

使用 Material 3 在 Compose 中設定主題

來源：<https://codelabs.developers.google.com/jetpack-compose-theming?hl=zh-tw>

步驟數：9

本程式碼研究室的目標是使用新推出的 Material Design 3 和 Material You 實作方法，示範如何在 Jetpack Compose 中設定主題。本指南與「DAC 指南：Compose 中的 Material Design 3」一致，並使用 Reply 範例應用程式進行示範。

目錄

1. [1. 簡介](#)
2. [2. 開始設定](#)
3. [3. Material 3 主題設定](#)
4. [4. 色彩配置](#)
5. [5. 在應用程式中新增動態色彩](#)
6. [6. 字體排版](#)
7. [7. 形狀](#)
8. [8. 強調效果](#)
9. [9. 恭喜](#)

1. 簡介

在這個程式碼研究室中，您可以瞭解如何使用 Material Design 3，在 Jetpack Compose 中設定應用程式的主題。此外，您也會學到 Material Design 3 色彩配置、字體排版和形狀的重要構成要素，協助您以個人化且易於使用的方式為應用程式設定主題。

此外，您也將探索如何支援動態主題設定，以及不同等級的強調效果。

注意：「Material Design 3」、「Material 3」和「M3」三個詞可互換。

學習目標

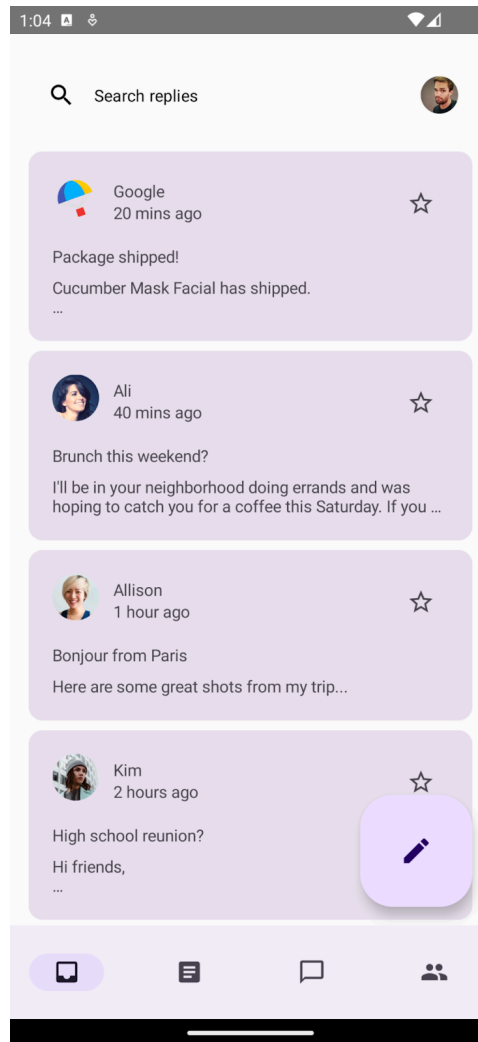
您可以在本程式碼研究室學到以下內容：

- Material 3 主題設定的重要注意事項
- Material 3 色彩配置，以及如何為應用程式產生主題
- 如何讓應用程式支援動態及淺色/深色主題設定
- 利用字體排版和圖形自訂應用程式
- Material 3 元件，以及如何自訂應用程式的樣式

建構內容

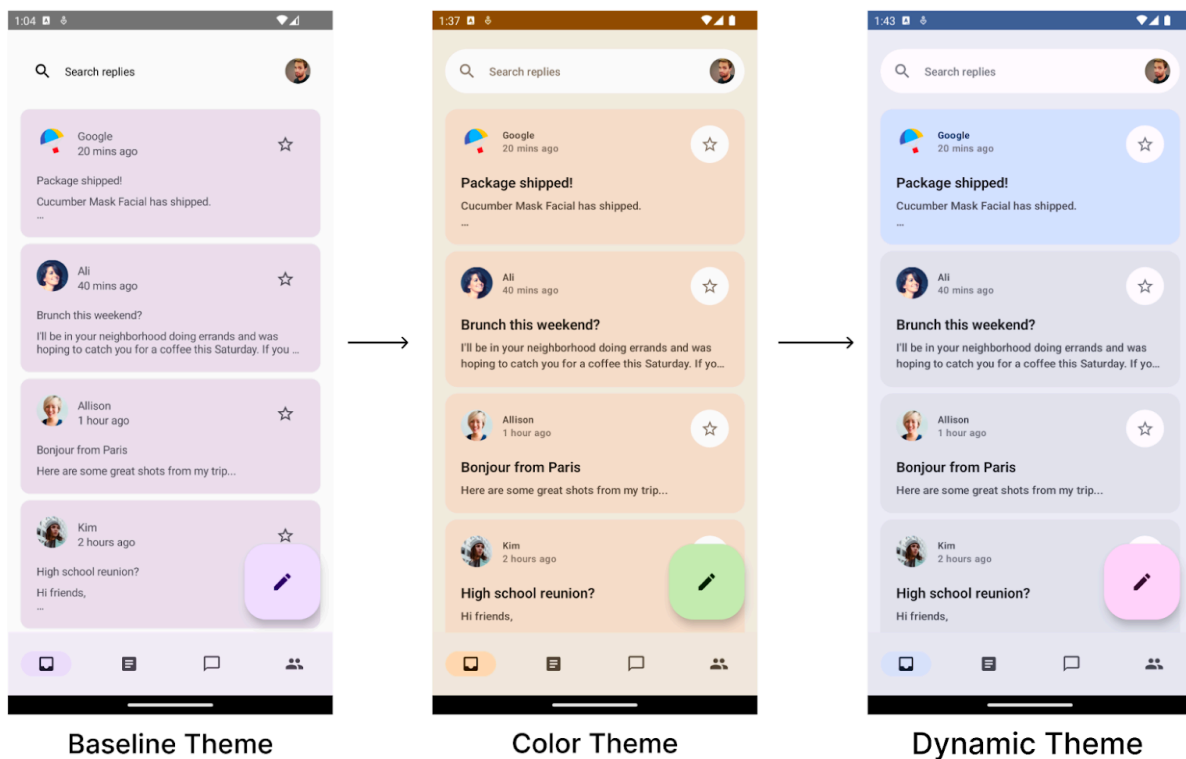
在本程式碼研究室中，您將對名為 Reply 的電子郵件用戶端應用程式設定主題。您會先從未設定樣式且採用基準主題的應用程式開始，接著運用所學知識為應用程式套用主題，並讓其支援深色主題。

注意：Material 預設為基準紫色配置。如未指定或自訂主題，Material 元件會改回使用基準主題。

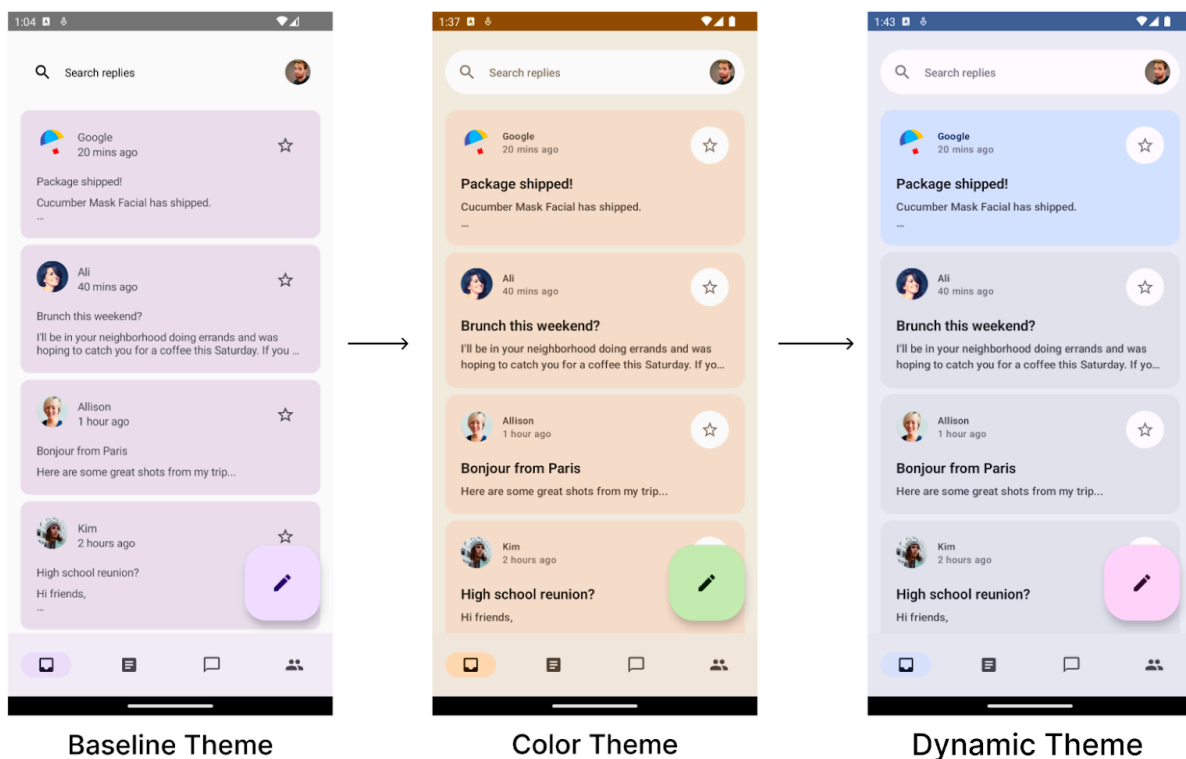


我們的應用程式預設起點為基準主題。

您要使用色彩配置、字體排版和形狀建立主題，然後將其套用至應用程式的電子郵件清單和詳細資料頁面。此外，您也將為應用程式新增動態主題支援。完成本程式碼研究室後，應用程式就能同時支援色彩主題和動態主題。



主題設定程式碼研究室的最終成果：套用淺色色彩主題設定與淺色動態主題設定的應用程式畫面。



主題設定程式碼研究室的最終成果：套用深色色彩主題設定與深色動態主題設定的應用程式畫面。

軟硬體需求

- 最新版 [Android Studio](#)
- 具備 [Kotlin 語言](#)的基本經驗

- 具備 [Jetpack Compose 基本知識](#)
 - 熟悉 Compose 版面配置的基本知識，例如[列](#)、[欄](#)和[修飾符](#)
-

2. 開始設定

在這個步驟中，您要下載 Reply 應用程式的完整程式碼，您將在本程式碼研究室中為該應用程式設定樣式。

取得程式碼

您可以在 [android-compose-codelabs](https://github.com/googlecodelabs/android-compose-codelabs) GitHub 存放區中找到本程式碼研究室的程式碼。如要複製該存放區，請執行下列命令：

```
$ git clone https://github.com/googlecodelabs/android-compose-codelabs
```

或者，您也可以下載兩個 ZIP 檔案：

- [範例](#)
- [解決方案](#)

查看範例應用程式

您剛才下載的程式碼包含所有 Compose 程式碼研究室可用的程式碼。如要完成本程式碼研究室，請在 Android Studio 中開啟 **ThemingCodelab** 專案。



- **themingCodelab**：包含本程式碼研究室起始和完成後程式碼的專案。

本專案是在多個 Git 分支版本中建構而成：



- **main**：包含本專案的範例程式碼；您必須修改程式碼，才能完成程式碼研究室



- **end**：包含本程式碼研究室完成後的程式碼。

建議您先從 main 分支版本的程式碼著手，依自己的步調逐步完成本程式碼研究室。您隨時可以變更專案的 Git 分支版本，以在 Android Studio 中執行任一版本。

探索範例程式碼

主要程式碼包含 UI 套件，其中含有您將與其互動的主要套件和檔案：

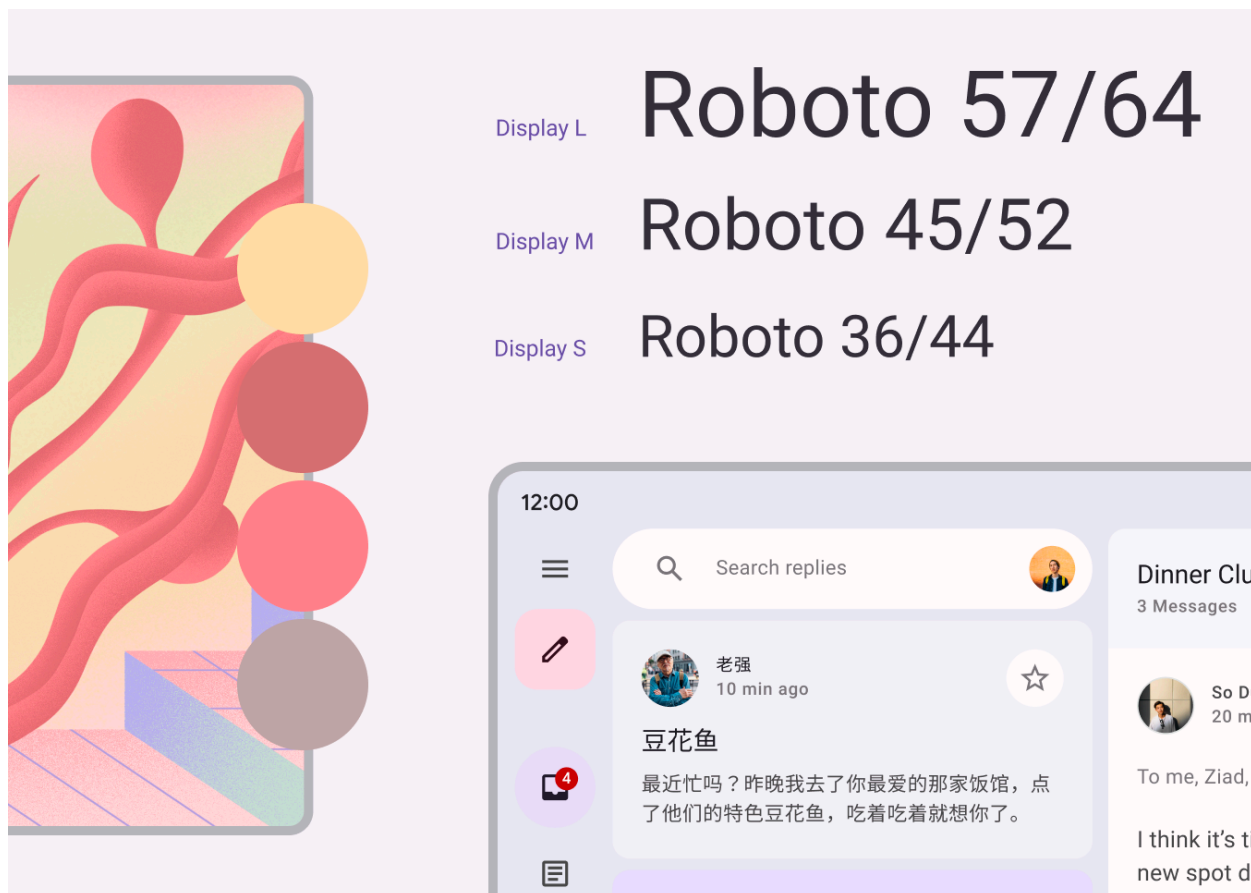
- `MainActivity.kt`：用於啟動 Reply 應用程式的進入點活動。
- `com.example.reply.ui.theme`：這個套件包含主題、字體排版和色彩配置。您將在此套件中新增 Material Design 主題設定。
- `com.example.reply.ui.components`：包含應用程式的自訂元件，例如清單項目、應用程式列等。您要為這些元件套用主題。
- `ReplyApp.kt`：這是我們主要的可組合函式，是 UI 樹狀結構的起點。您將在這個檔案中套用頂層主題設定。

本程式碼研究室著重於 `ui` 套件檔案。

3. Material 3 主題設定

Jetpack Compose 提供 [Material Design](#) 這項產品，這是用於建立數位介面的全方位設計系統。Material Design [元件](#) (按鈕、資訊卡、切換鈕等) 是以 [Material Design 主題設定](#) 為基礎建構而成，能夠以系統化的方式自訂 Material Design，以更切合您產品品牌的方式呈現。

Material 3 主題包含下列子系統，可在應用程式中加入主題：[色彩配置](#)、[字體排版](#)和[形狀](#)。自訂這些值時，系統會自動在您用於建構應用程式的 M3 元件中反映變更。讓我們深入探討各項子系統，並在範例應用程式中實作。

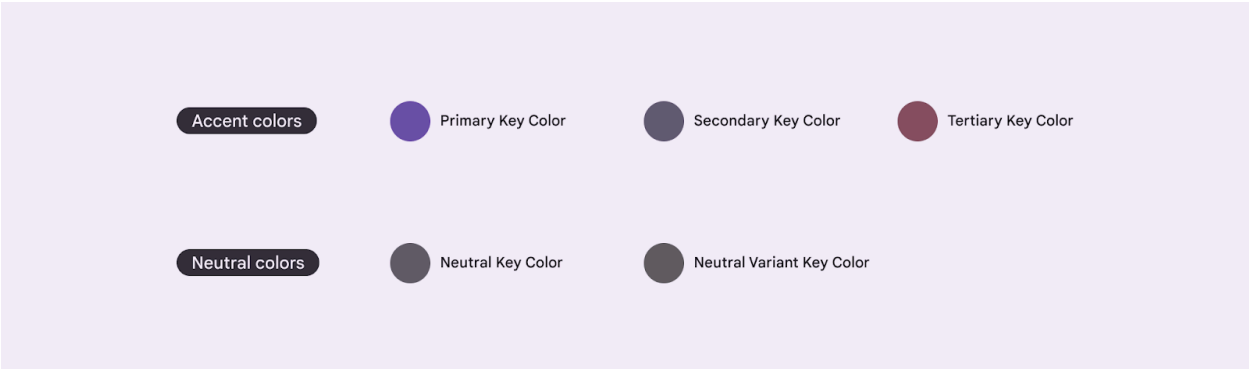


Material 3 的子系統有顏色、字體排版和形狀。

步驟 4 · 約 9 分鐘

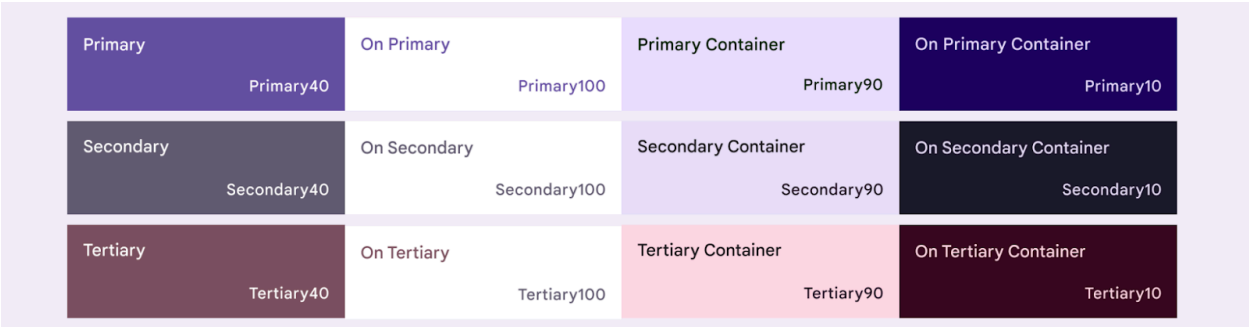
4. 色彩配置

色彩配置的基礎是五組主要顏色，每組皆與 Material 3 元件所用 13 種色調的調色盤相關。



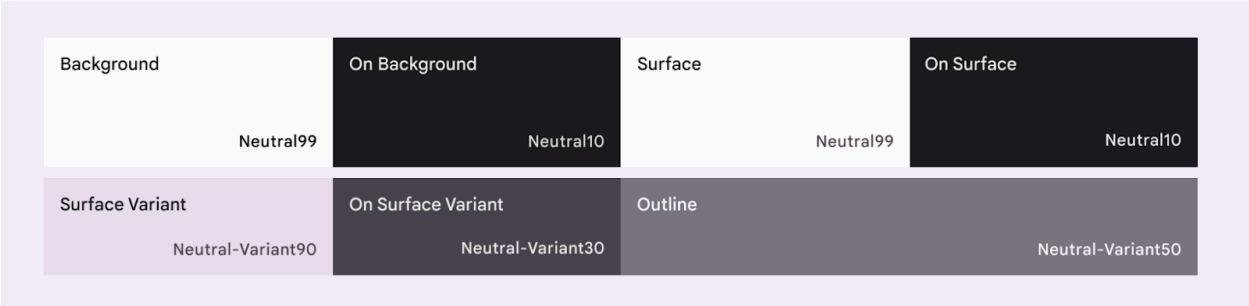
建立 M3 主題設定的五種主要顏色。

每個強調色 (原色、二次色及三次色) 都會提供四種不同色調的相容顏色，可用於配色、定義強調效果及呈現視覺效果。



原色、二次色及三次色基準強調色的四種色調顏色。

同樣地，中性色也可分為四種相同色調，可用於表面及背景。在將文字圖示放置在任何表面時，這些顏色對於強調效果而言也十分重要。



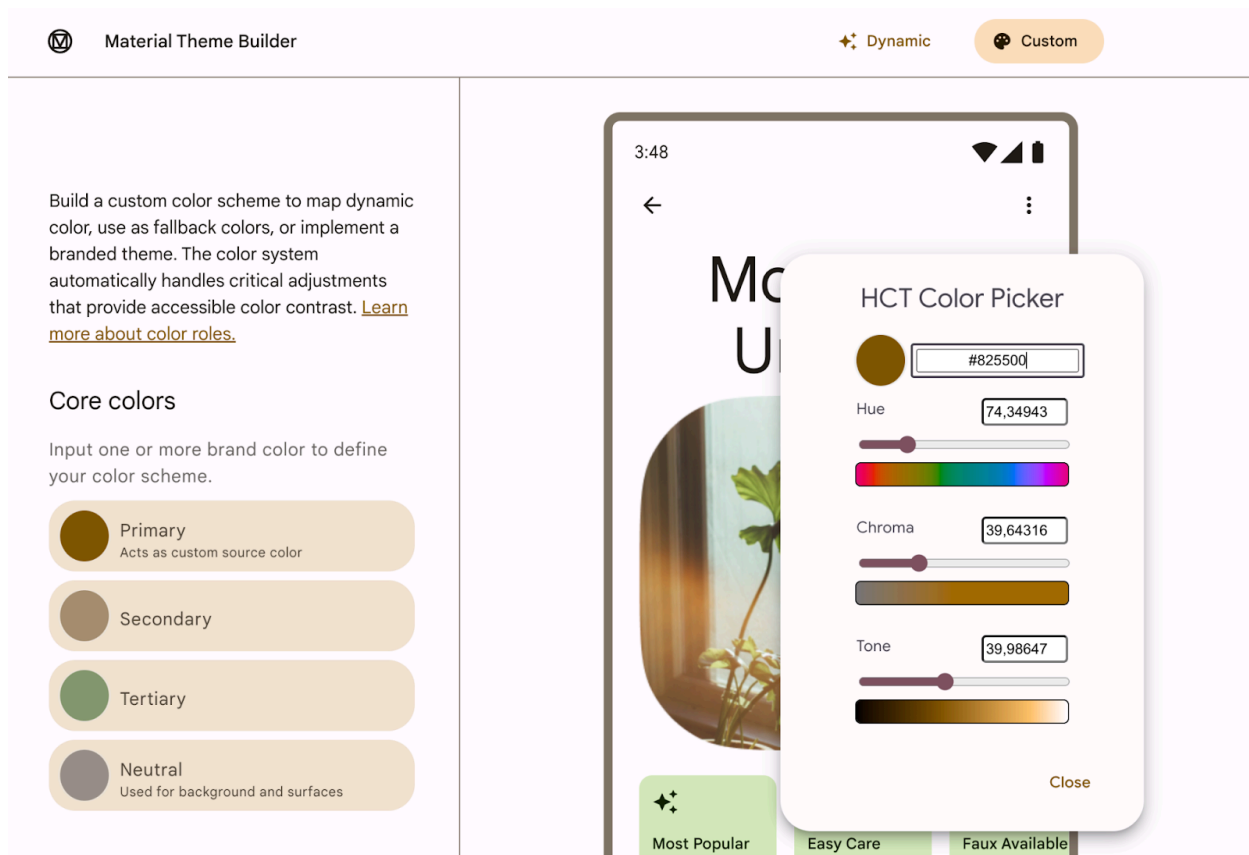
基準中性色的四種色調顏色。

如想進一步瞭解色彩配置和顏色角色，請前往[這個網頁](#)。

產生色彩配置

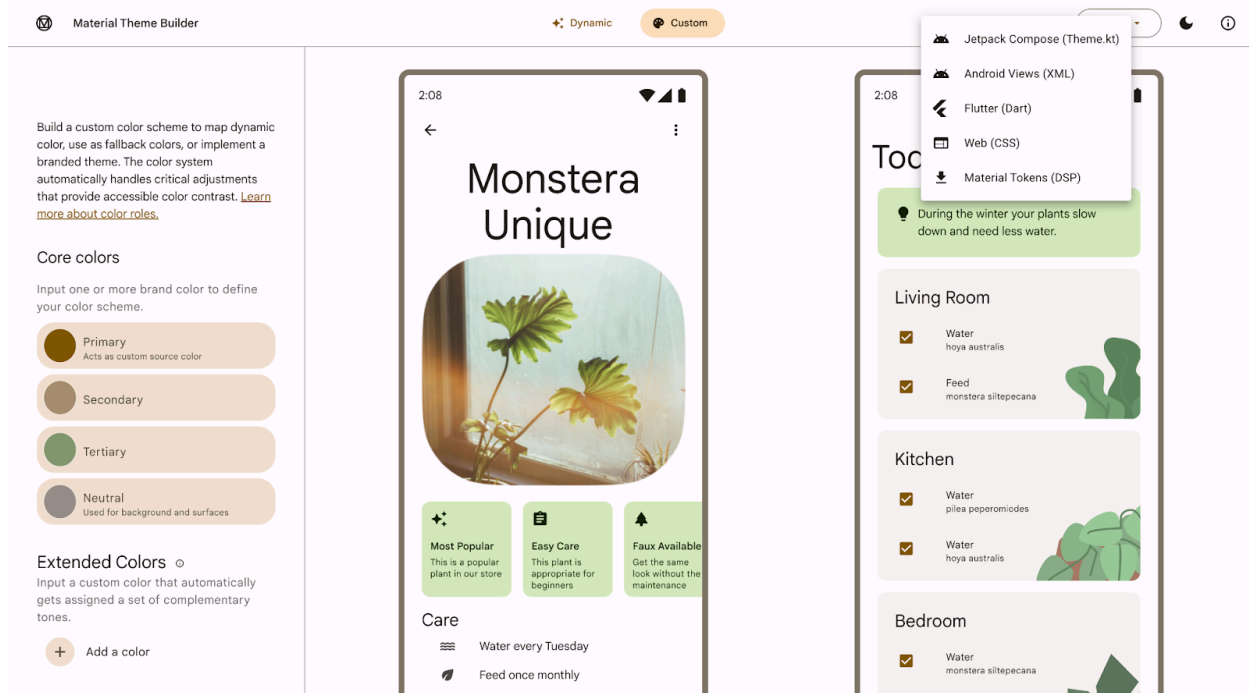
雖然您可以手動建立自訂 [ColorScheme](#)，但使用品牌的來源色彩來產生這個類別是較簡單的做法。您可以使用 [Material Design 主題建構工具](#) 執行這項操作，並視需要匯出 Compose 主題設定程式碼。

您可以選擇任何想要的顏色，但在本範例中，您將使用預設的「Reply」原色 #825500。請在左側的「Core colors」部分按一下「Primary」顏色，然後在顏色挑選器中新增程式碼。



在 Material 主題建構工具中新增原色代碼。

在 Material 主題建構工具中新增原色後，您會看到以下主題和右上角的匯出選項。在本程式碼研究室中，您要匯出 Jetpack Compose 中的主題。



Material 主題建構工具，畫面右上角有匯出選項。

原色 #825500 會產生要新增至應用程式的下列主題。Material 3 提供各種顏色角色，可靈活呈現元件的狀態、顯眼程度和強調效果。

Light Scheme			
Primary	On Primary	Primary Container	On Primary Container
Secondary	On Secondary	Secondary Container	On Secondary Container
Tertiary	On Tertiary	Tertiary Container	On Tertiary Container
Error	On Error	Error Container	On Error Container
Background	On Background	Surface	On Surface
Outline		Surface-Variant	On Surface-Variant
Dark Scheme			
Primary	On Primary	Primary Container	On Primary Container
Secondary	On Secondary	Secondary Container	On Secondary Container
Tertiary	On Tertiary	Tertiary Container	On Tertiary Container
Error	On Error	Error Container	On Error Container
Background	On Background	Surface	On Surface
Outline		Surface-Variant	On Surface-Variant

從原色匯出的淺色和深色色彩配置。

`The Color.kt` 產生的檔案包含主題的顏色，以及針對淺色和深色主題顏色定義的所有角色。

Color.kt


```
package com.example.reply.ui.theme

import androidx.compose.ui.graphics.Color

val md_theme_light_primary = Color(0xFF825500)
val md_theme_light_onPrimary = Color(0xFFFFFFFF)
val md_theme_light_primaryContainer = Color(0xFFFFDDB3)
val md_theme_light_onPrimaryContainer = Color(0xFF291800)
val md_theme_light_secondary = Color(0xFF6F5B40)
val md_theme_light_onSecondary = Color(0xFFFFFFFF)
val md_theme_light_secondaryContainer = Color(0xFFBDEBC)
val md_theme_light_onSecondaryContainer = Color(0xFF271904)
val md_theme_light_tertiary = Color(0xFF51643F)
val md_theme_light_onTertiary = Color(0xFFFFFFFF)
val md_theme_light_tertiaryContainer = Color(0xFFD4EAB)
val md_theme_light_onTertiaryContainer = Color(0xFF102004)
val md_theme_light_error = Color(0xFFBA1A1A)
val md_theme_light_errorContainer = Color(0xFFFFDAD6)
val md_theme_light_onError = Color(0xFFFFFFFF)
val md_theme_light_onErrorContainer = Color(0xFF410002)
val md_theme_light_background = Color(0xFFFFFBBF)
val md_theme_light_onBackground = Color(0xFF1F1B16)
val md_theme_light_surface = Color(0xFFFFFBBF)
val md_theme_light_onSurface = Color(0xFF1F1B16)
val md_theme_light_surfaceVariant = Color(0xFFFF0E0CF)
val md_theme_light_onSurfaceVariant = Color(0xFF4F4539)
val md_theme_light_outline = Color(0xFF817567)
val md_theme_light_inverseOnSurface = Color(0xFFF9EFE7)
val md_theme_light_inverseSurface = Color(0xFF34302A)
val md_theme_light_inversePrimary = Color(0xFFFFFB951)
val md_theme_light_shadow = Color(0xFF000000)
val md_theme_light_surfaceTint = Color(0xFF825500)
val md_theme_light_outlineVariant = Color(0xFFD3C4B4)
val md_theme_light_scrim = Color(0xFF000000)

val md_theme_dark_primary = Color(0xFFFFFB951)
val md_theme_dark_onPrimary = Color(0xFF452B00)
val md_theme_dark_primaryContainer = Color(0xFF633F00)
val md_theme_dark_onPrimaryContainer = Color(0xFFFFDDB3)
val md_theme_dark_secondary = Color(0xFFDDC2A1)
val md_theme_dark_onSecondary = Color(0xFF3E2D16)
val md_theme_dark_secondaryContainer = Color(0xFF56442A)
val md_theme_dark_onSecondaryContainer = Color(0xFFBDEBC)
val md_theme_dark_tertiary = Color(0xFFB8CEA1)
val md_theme_dark_onTertiary = Color(0xFF243515)
val md_theme_dark_tertiaryContainer = Color(0xFF3A4C2A)
val md_theme_dark_onTertiaryContainer = Color(0xFFD4EAB)
val md_theme_dark_error = Color(0xFFFFB4AB)
val md_theme_dark_errorContainer = Color(0xFF93000A)
val md_theme_dark_onError = Color(0xFF690005)
val md_theme_dark_onErrorContainer = Color(0xFFFFDAD6)
val md_theme_dark_background = Color(0xFF1F1B16)
val md_theme_dark_onBackground = Color(0xFFEAE1D9)
```

```
val md_theme_dark_surface = Color(0xFF1F1B16)
val md_theme_dark_onSurface = Color(0xFFEAE1D9)
val md_theme_dark_surfaceVariant = Color(0xFF4F4539)
val md_theme_dark_onSurfaceVariant = Color(0xFFD3C4B4)
val md_theme_dark_outline = Color(0xFF9C8F80)
val md_theme_dark_inverseOnSurface = Color(0xFF1F1B16)
val md_theme_dark_inverseSurface = Color(0xFFEAE1D9)
val md_theme_dark_inversePrimary = Color(0xFF825500)
val md_theme_dark_shadow = Color(0xFF000000)
val md_theme_dark_surfaceTint = Color(0xFFFFB951)
val md_theme_dark_outlineVariant = Color(0xFF4F4539)
val md_theme_dark_scrim = Color(0xFF000000)

val seed = Color(0xFF825500)
```

注意：您也可以從匯出的 `Color.kt` 檔案中複製及貼上程式碼，但請務必將程式碼中的套件名稱從 `com.example.compose` 變更為 `com.example.reply.ui.theme`，以免匯入錯誤。

The `Theme.kt` 產生的檔案含有淺色和深色色彩配置，以及應用程式主題。此外也包含了主要主題設定的可組合函式 `AppTheme()`。

Theme.kt

```

package com.example.reply.ui.theme

import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.lightColorScheme
import androidx.compose.material3.darkColorScheme
import androidx.compose.runtime.Composable

private val LightColors = lightColorScheme(
    primary = md_theme_light_primary,
    onPrimary = md_theme_light_onPrimary,
    primaryContainer = md_theme_light_primaryContainer,
    onPrimaryContainer = md_theme_light_onPrimaryContainer,
    secondary = md_theme_light_secondary,
    onSecondary = md_theme_light_onSecondary,
    secondaryContainer = md_theme_light_secondaryContainer,
    onSecondaryContainer = md_theme_light_onSecondaryContainer,
    tertiary = md_theme_light_tertiary,
    onTertiary = md_theme_light_onTertiary,
    tertiaryContainer = md_theme_light_tertiaryContainer,
    onTertiaryContainer = md_theme_light_onTertiaryContainer,
    error = md_theme_light_error,
    errorContainer = md_theme_light_errorContainer,
    onError = md_theme_light_onError,
    onErrorContainer = md_theme_light_onErrorContainer,
    background = md_theme_light_background,
    onBackground = md_theme_light_onBackground,
    surface = md_theme_light_surface,
    onSurface = md_theme_light_onSurface,
    surfaceVariant = md_theme_light_surfaceVariant,
    onSurfaceVariant = md_theme_light_onSurfaceVariant,
    outline = md_theme_light_outline,
    inverseOnSurface = md_theme_light_inverseOnSurface,
    inverseSurface = md_theme_light_inverseSurface,
    inversePrimary = md_theme_light_inversePrimary,
    surfaceTint = md_theme_light_surfaceTint,
    outlineVariant = md_theme_light_outlineVariant,
    scrim = md_theme_light_scrim,
)

private val DarkColors = darkColorScheme(
    primary = md_theme_dark_primary,
    onPrimary = md_theme_dark_onPrimary,
    primaryContainer = md_theme_dark_primaryContainer,
    onPrimaryContainer = md_theme_dark_onPrimaryContainer,
    secondary = md_theme_dark_secondary,
    onSecondary = md_theme_dark_onSecondary,
    secondaryContainer = md_theme_dark_secondaryContainer,
    onSecondaryContainer = md_theme_dark_onSecondaryContainer,
    tertiary = md_theme_dark_tertiary,

```

```

onTertiary = md_theme_dark_onTertiary,
tertiaryContainer = md_theme_dark_tertiaryContainer,
onTertiaryContainer = md_theme_dark_onTertiaryContainer,
error = md_theme_dark_error,
errorContainer = md_theme_dark_errorContainer,
onError = md_theme_dark_onError,
onErrorContainer = md_theme_dark_onErrorContainer,
background = md_theme_dark_background,
onBackground = md_theme_dark_onBackground,
surface = md_theme_dark_surface,
onSurface = md_theme_dark_onSurface,
surfaceVariant = md_theme_dark_surfaceVariant,
onSurfaceVariant = md_theme_dark_onSurfaceVariant,
outline = md_theme_dark_outline,
inverseOnSurface = md_theme_dark_inverseOnSurface,
inverseSurface = md_theme_dark_inverseSurface,
inversePrimary = md_theme_dark_inversePrimary,
surfaceTint = md_theme_dark_surfaceTint,
outlineVariant = md_theme_dark_outlineVariant,
scrim = md_theme_dark_scrim,
)

@Composable
fun AppTheme(
    useDarkTheme: Boolean = isSystemInDarkTheme(),
    content: @Composable() () -> Unit
) {
    val colors = if (!useDarkTheme) {
        LightColors
    } else {
        DarkColors
    }

    MaterialTheme(
        colorScheme = colors,
        content = content
    )
}

```

注意：您也可以從匯出的 `Theme.kt` 檔案中複製及貼上程式碼，但請務必將程式碼中的套件名稱從 `com.example.compose` 變更為 `com.example.reply.ui.theme`，以免匯入錯誤。

在 Jetpack Compose 中主題設定實作的核心元素是 `MaterialTheme` 可組合函式。

您可以將 `MaterialTheme()` 可組合函式納入 `AppTheme()` 函式中，後者會採用兩個參數：

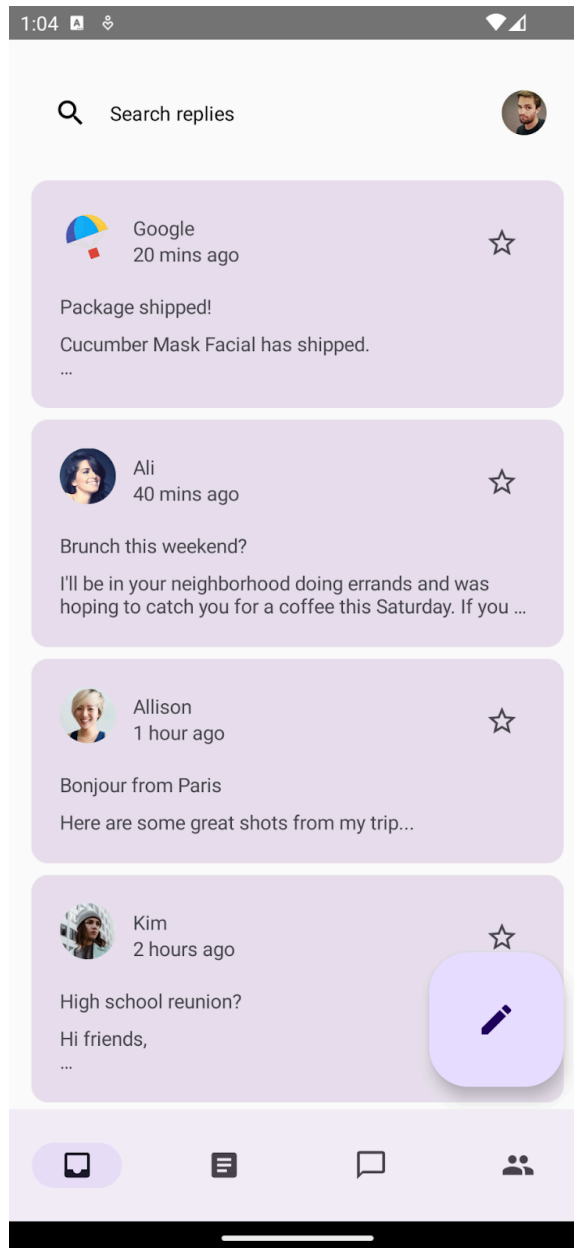
- `useDarkTheme` - 這個參數會與 `isSystemInDarkTheme()` 函式相關聯，可觀察系統的主題設定，並套用淺色或深色主題。如果您想以手動方式讓應用程式呈現淺色或深色主題，可以將布林值傳遞至 `useDarkTheme`。
- `content`：要套用主題的內容。

Theme.kt

```
@Composable
fun AppTheme(
    useDarkTheme: Boolean = isSystemInDarkTheme(),
    content: @Composable() () -> Unit
) {
    val colors = if (!useDarkTheme) {
        LightColors
    } else {
        DarkColors
    }

    MaterialTheme(
        colorScheme = colors,
        content = content
    )
}
```

如果您現在嘗試執行應用程式，應該會看到相同情況。即使您已經匯入含有新主題顏色的新色彩配置，您看到的依然會是基準主題設定，因為您尚未將主題套用至 Compose 應用程式。



未套用任何主題，採用基準主題設定的應用程式。

如要套用新主題，請在 `MainActivity.kt` 中，將主要主題函式 `AppTheme()` 納入主要可組合函式 `ReplyApp`。

MainActivity.kt

```
setContent {  
    val uiState by viewModel.uiState.collectAsStateWithLifecycle()  
  
    AppTheme {  
        ReplyApp(/*...*/)  
    }  
}
```

此外，您也可以更新預覽函式，查看應用程式預覽畫面所套用的主題。請使用 `AppTheme` 將 `ReplyApp` 可組合函式納入 `ReplyAppPreview()`，以便為預覽畫面套用主題設定

您在兩個預覽參數中定義了淺色和深色系統主題，因此您會看到兩種預覽畫面。

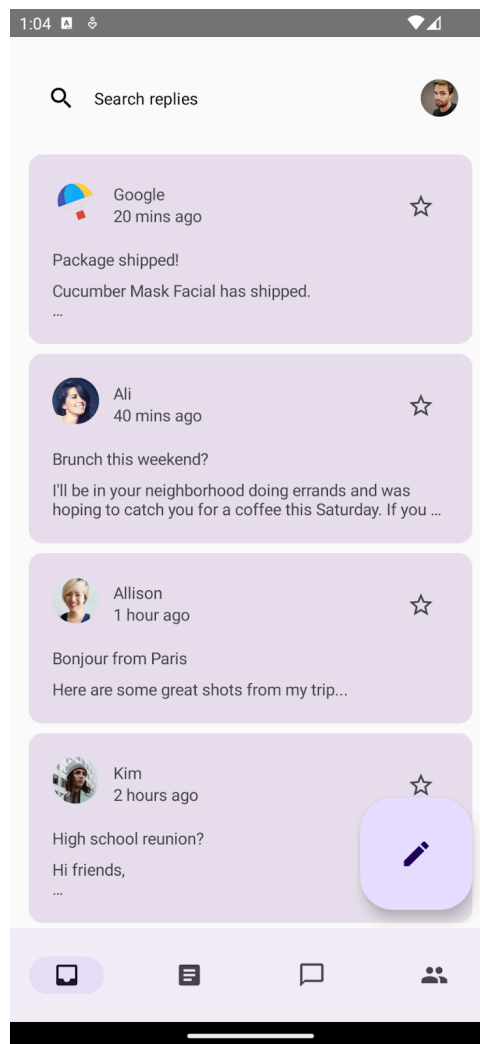
MainActivity.kt

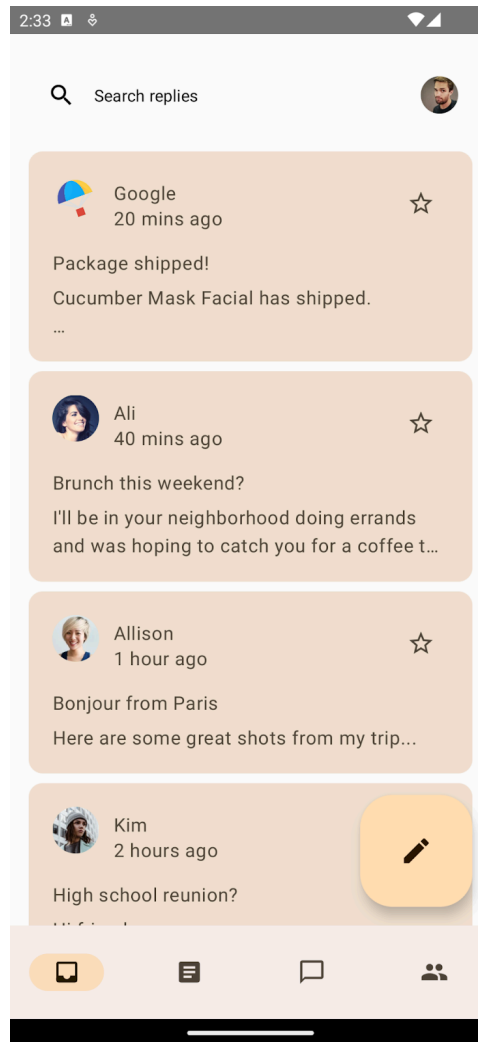
```

@Preview(
    uiMode = Configuration.UI_MODE_NIGHT_YES,
    name = "DefaultPreviewDark"
)
@Preview(
    uiMode = Configuration.UI_MODE_NIGHT_NO,
    name = "DefaultPreviewLight"
)
@Composable
fun ReplyAppPreview() {
    AppTheme {
        ReplyApp(
            replyHomeUIState = ReplyHomeUIState(
                emails = LocalEmailsDataProvider.allEmails
            )
        )
    }
}

```

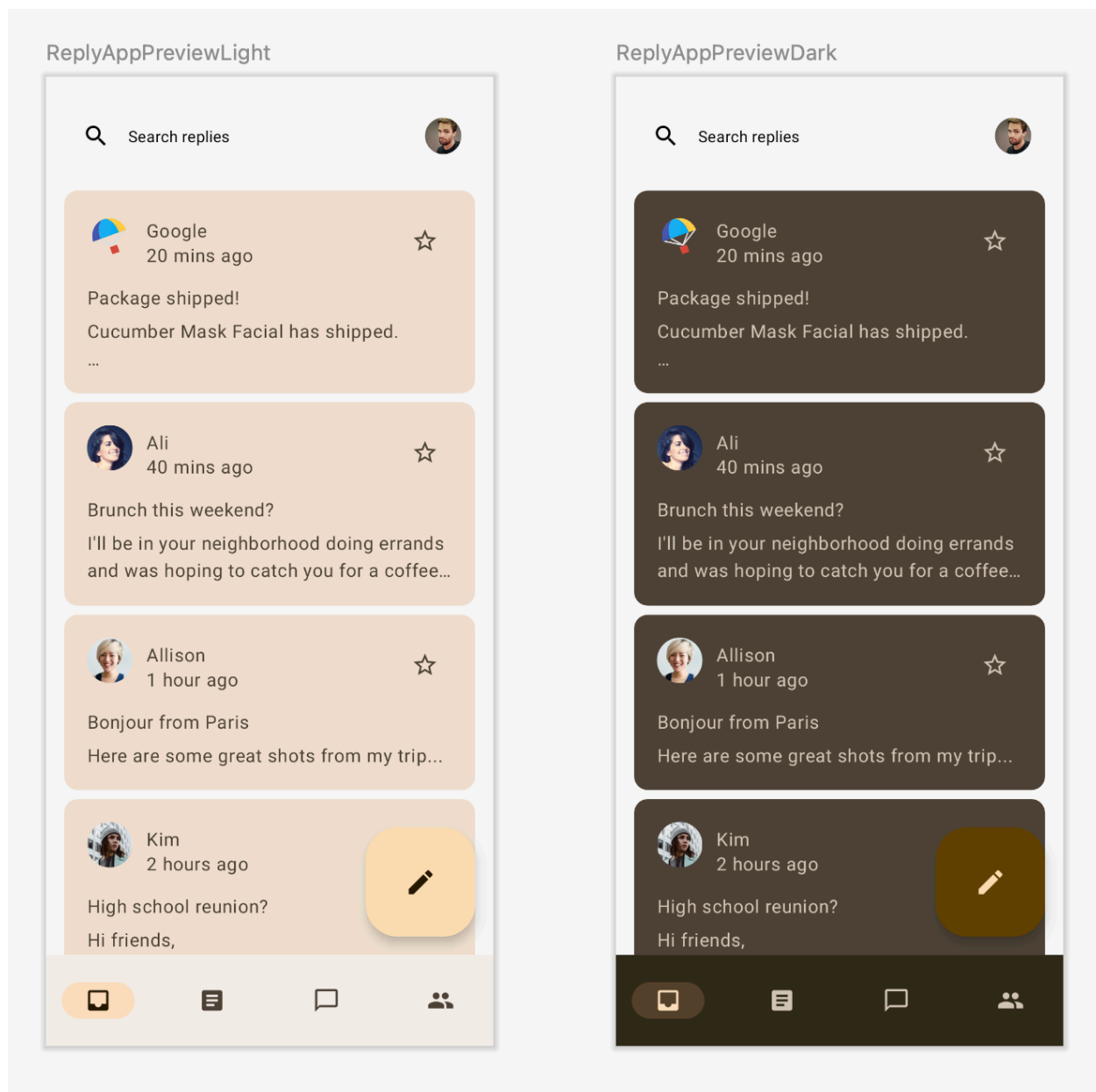
如果您現在執行應用程式，應會看到應用程式預覽畫面採用了已匯入主題的顏色，而非基準主題。





採用基準主題的應用程式 (左圖)。

採用匯入色彩主題的應用程式 (右圖)。



採用匯入色彩主題的淺色及深色應用程式預覽畫面

Material 3 支援淺色和深色彩色配置。您只會納入採用匯入主題的應用程式；Material 3 元件使用的是預設顏色角色。開始在應用程式中新增顏色角色之前，讓我們先瞭解顏色角色的概念及用法。

顏色角色和無障礙設計

每個顏色角色都可以用在各種位置，視元件的狀態、顯眼程度和強調效果而定。

Primary	On Primary	Primary Container	On Primary Container
Secondary	On Secondary	Secondary Container	On Secondary Container
Tertiary	On Tertiary	Tertiary Container	On Tertiary Container

原色、二次色及三次色顏色角色。

「Primary」是基本顏色，用於主要元件，例如顯眼的按鈕和啟用中狀態。

「Secondary」主要顏色是用於 UI 中較不顯眼的元件，例如篩選器方塊。

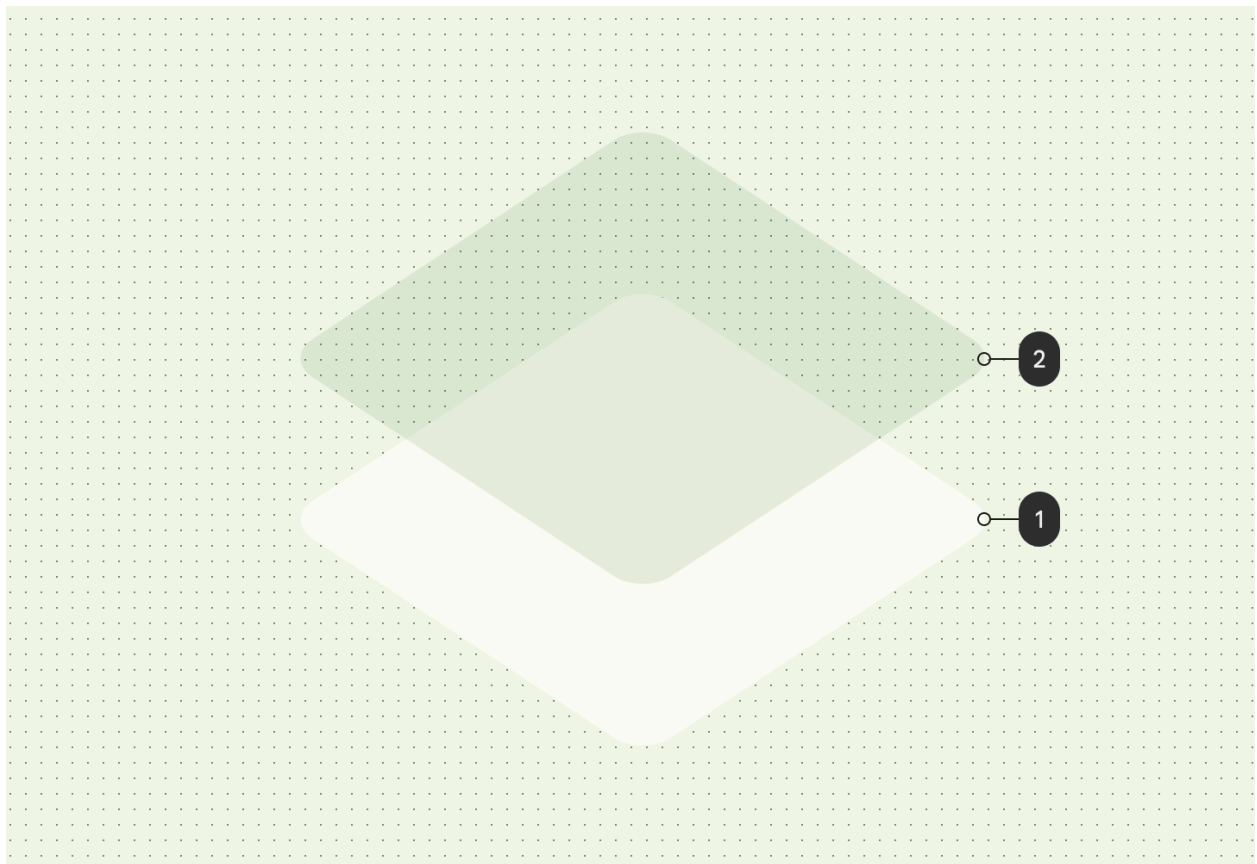
「Tertiary」主要顏色是用於提供對比色，以及用於應用程式背景和表面的中性色。

Material 的色彩系統提供標準的**色調值**和測量方法，可用於達到一目瞭然的對比度。在原色上使用 on-primary，在 primary-container 上使用 on-primary-container，並對其他強調色和中性色採取相同做法，為使用者提供清楚易懂的顏色對比。

詳情請參閱「[顏色角色和無障礙設計](#)」。

色調與陰影高度

Material 3 主要使用色調色彩重疊來表示高度。這是區分容器和表面的新方式，除了使用陰影之外，還使用更加顯眼的色調來提高色調高度。



等級 2 的色調高度，會從主要色彩運算單元取得色彩。

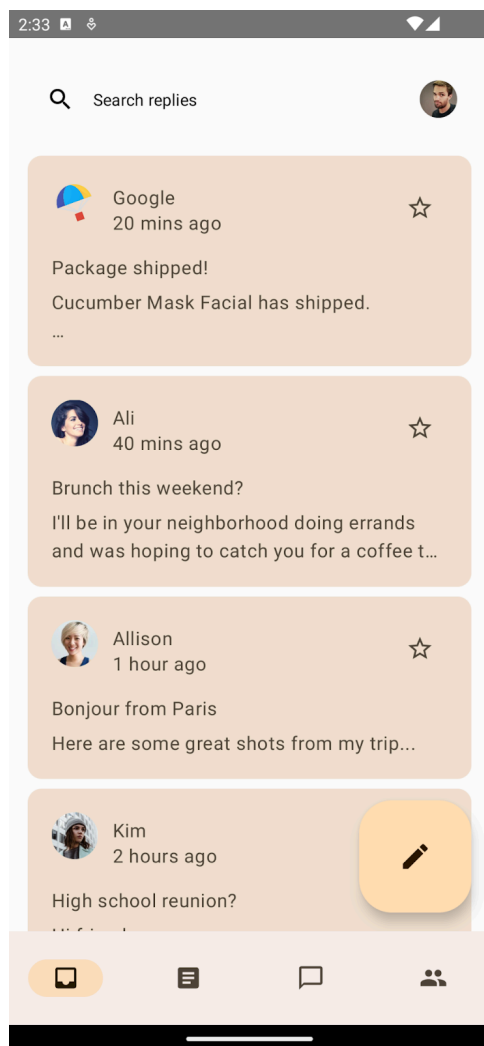
深色主題中的高度重疊在 Material Design 3 中也已改為色調色彩重疊。重疊顏色來自主要色彩運算單元。

M3 **表面**是大多數 M3 元件的備用可組合函式，同時支援色調及陰影高度：

```
Surface(  
  modifier = modifier,  
  tonalElevation = {...}  
  shadowElevation = {...}  
) {  
  Column(content = content)  
}
```

新增顏色至應用程式

如果您執行應用程式，會看到應用程式中顯示了匯出的顏色，其中元件使用的是預設顏色。現在我們已經瞭解顏色角色的概念和用法，接著要為應用程式設定正確的顏色角色主題。



含有顏色主題的應用程式，元件採用的是預設顏色角色。

表面顏色

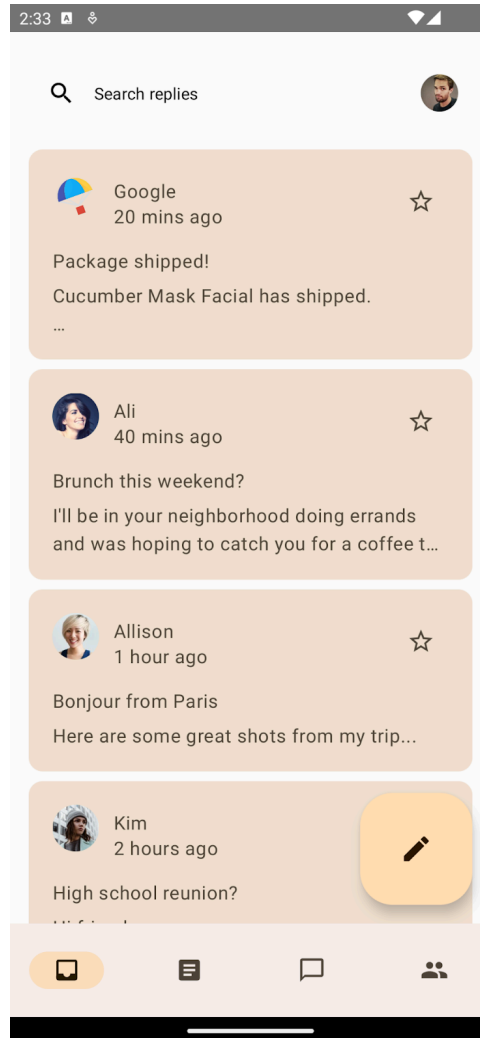
在主畫面中，您要將主應用程式可組合函式納入 `Surface()` 中，以便提供要放置應用程式內容的表面。開啟 `MainActivity.kt`，並使用 `Surface` 納入 `ReplyApp()` 可組合函式。

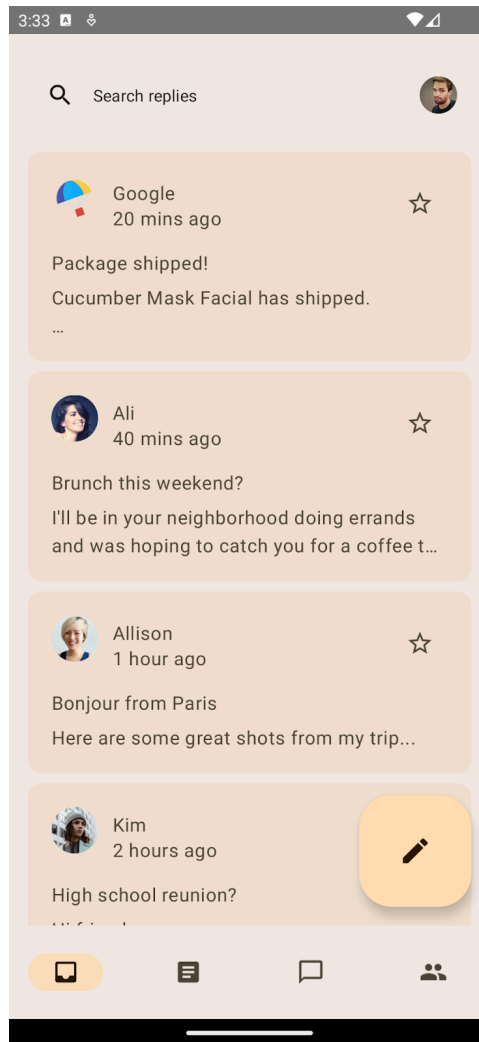
您也將提供 5.dp 的色調高度，為表面提供主要運算單元的色調顏色，以便與清單項目及其頂端的搜尋列形成對比。根據預設，表面的色調和陰影高度為 0.dp。

MainActivity.kt

```
AppTheme {
  Surface(tonalElevation = 5.dp) {
    ReplyApp(
      replyHomeUIState = uiState,
      // other parameters
    )
  }
}
```

如果您現在執行應用程式，並查看「List」和「Detail」頁面，應該會看到色調表面已套用至整個應用程式。



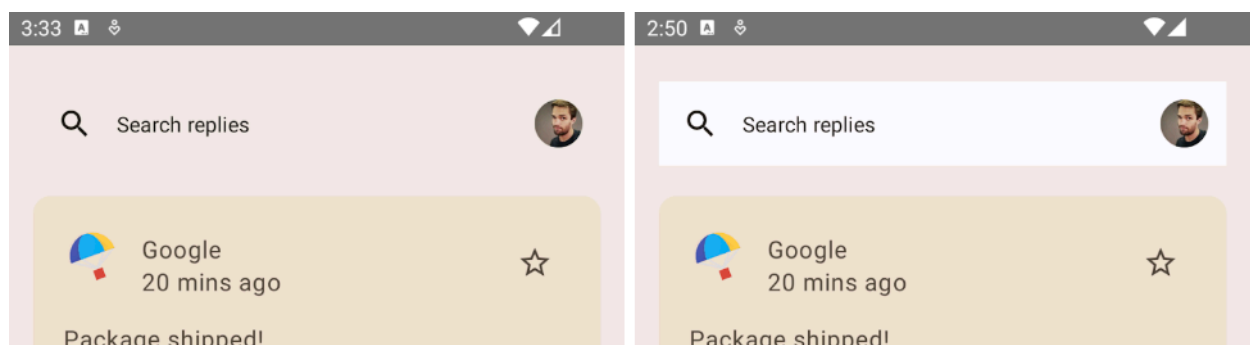


沒有表面和色調顏色的應用程式背景 (左圖)。

已套用表面和色調顏色的應用程式背景 (右圖)。

應用程式列顏色

我們頂端的自訂搜尋列在設計時並未要求明確背景。根據預設，它會改回預設的基礎表面。您可以提供背景來做出明確區別。



無背景的自訂搜尋列 (左圖)。

有背景的自訂搜尋列 (右圖)。

現在您將編輯含有應用程式列的 `ui/components/ReplyAppBars.kt`。您要將 `MaterialTheme.colorScheme.background` 新增至 `Row` 可組合函式的 `Modifier`。

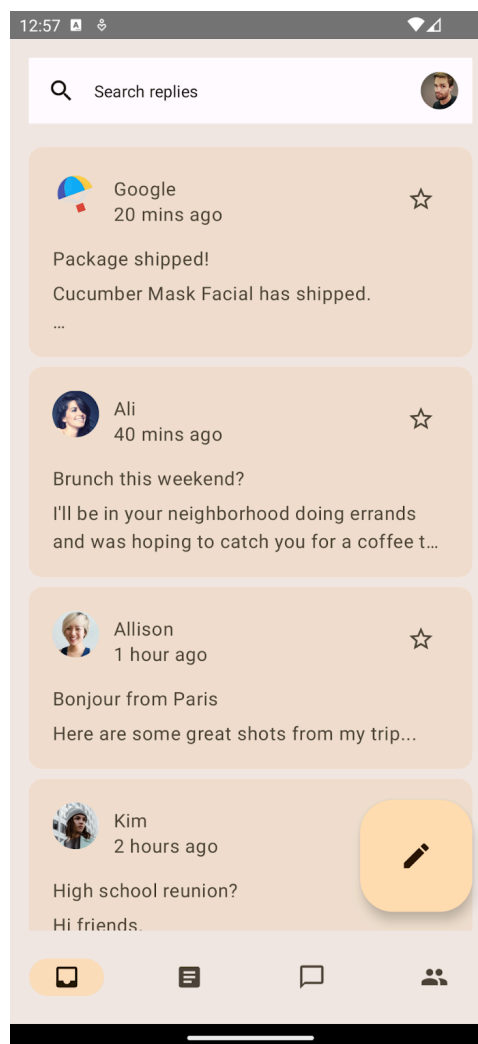
ResponseAppBars.kt

```

@Composable
fun ReplySearchBar(modifier: Modifier = Modifier) {
    Row(
        modifier = modifier
            .fillMaxWidth()
            .padding(16.dp)
            .background(MaterialTheme.colorScheme.background),
        verticalAlignment = Alignment.CenterVertically
    ) {
        // Search bar content
    }
}

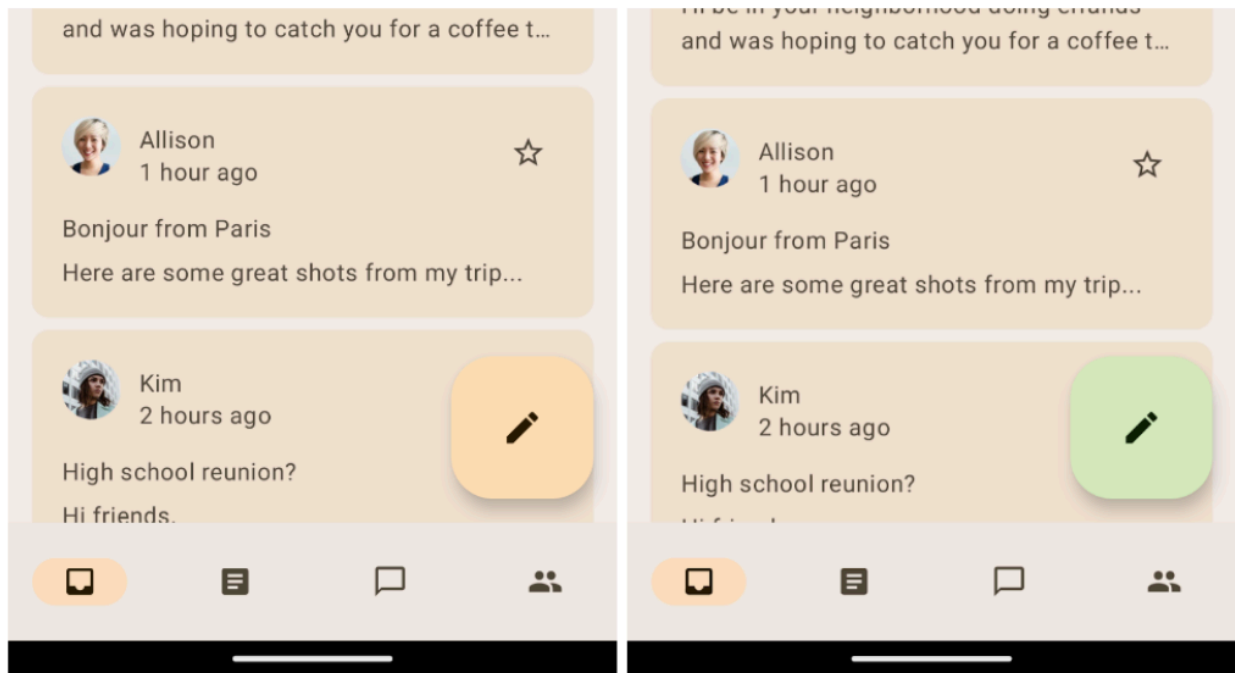
```

現在，您應該會看到色調表面與套用背景顏色的應用程式列之間有明確區別。



位於色調表面頂端，含有背景顏色的搜尋列。

懸浮動作按鈕顏色



未套用任何主題的大型懸浮動作按鈕 (FAB) (左圖)。

套用三次色主題設定的大型懸浮動作按鈕 (FAB) (右圖)。

在主畫面上，您可以凸顯懸浮動作按鈕 (FAB) 的外觀，讓它成為更顯眼的行動號召按鈕。如要實作這個效果，請為該按鈕套用第三色強調色。

在 `ReplyListContent.kt` 檔案中，將 FAB 的 `containerColor` 更新為 `tertiaryContainer` 顏色，並將內容顏色更新為 `onTertiaryContainer`，以維持無障礙設計和顏色對比度。

ReplyListContent.kt

```
ReplyInboxScreen(/*...*/) {  
    // Email list content  
    LargeFloatingActionButton(  
        containerColor = MaterialTheme.colorScheme.tertiaryContainer,  
        contentColor = MaterialTheme.colorScheme.onTertiaryContainer  
    ){  
        /*...*/  
    }  
}
```

執行應用程式，查看套用主題的懸浮動作按鈕 (FAB)。在本程式碼研究室中，您使用的是 `LargeFloatingActionButton`。

注意：Material 3 設計系統提供多個 [FAB 變化版本](#) 供您查看。

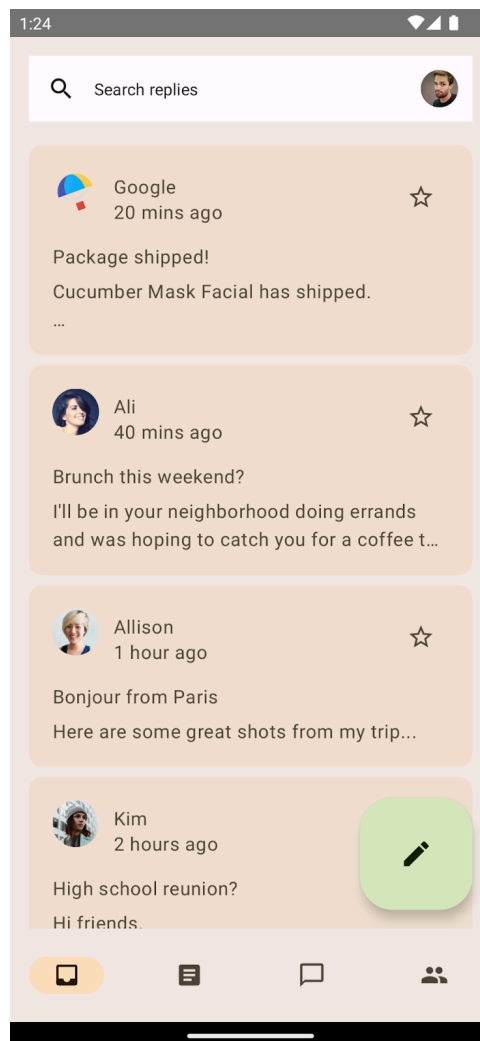
資訊卡顏色

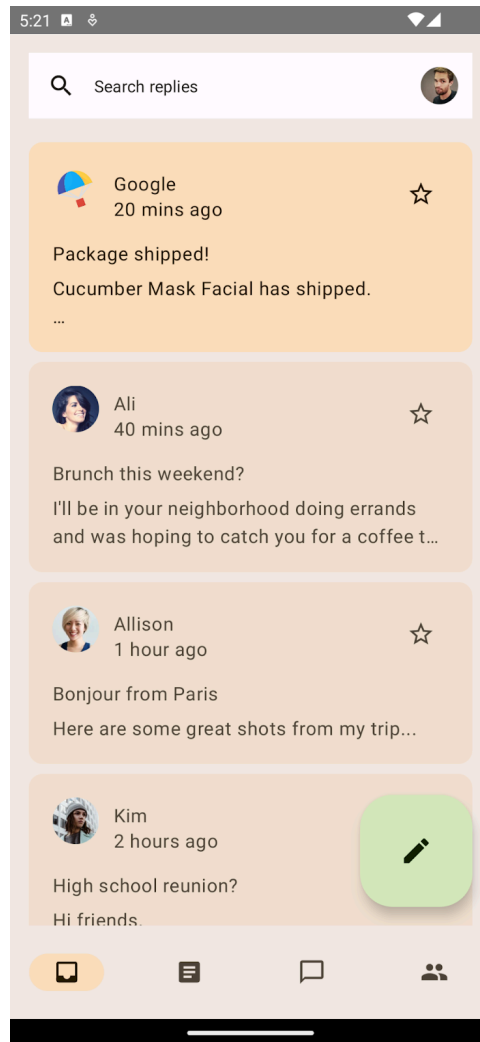
主畫面上的電子郵件清單使用的是資訊卡元件。根據預設，它是 [填充資訊卡](#)，會使用表面變化版本顏色做為容器顏色，藉此明確區分表面和資訊卡的顏色。Compose 也提供 `ElevatedCard` 和 `OutlinedCard` 的實作。

您還可以提供第二次色調，進一步凸顯重要的部分項目。針對重要電子郵件，您要使用 `CardDefaults.cardColors()` 更新資訊卡容器顏色，藉此修改 `ui/components/ReplyEmailListItem.kt`：

ReplyEmailListItem.kt

```
Card(
    modifier = modifier
        .padding(horizontal = 16.dp, vertical = 4.dp)
        .semantics { selected = isSelected }
        .clickable { navigateToDetail(email.id) },
    colors = CardDefaults.cardColors(
        containerColor = if (email.isImportant)
            MaterialTheme.colorScheme.secondaryContainer
        else MaterialTheme.colorScheme.surfaceVariant
    )
){
    /*...*/
}
```

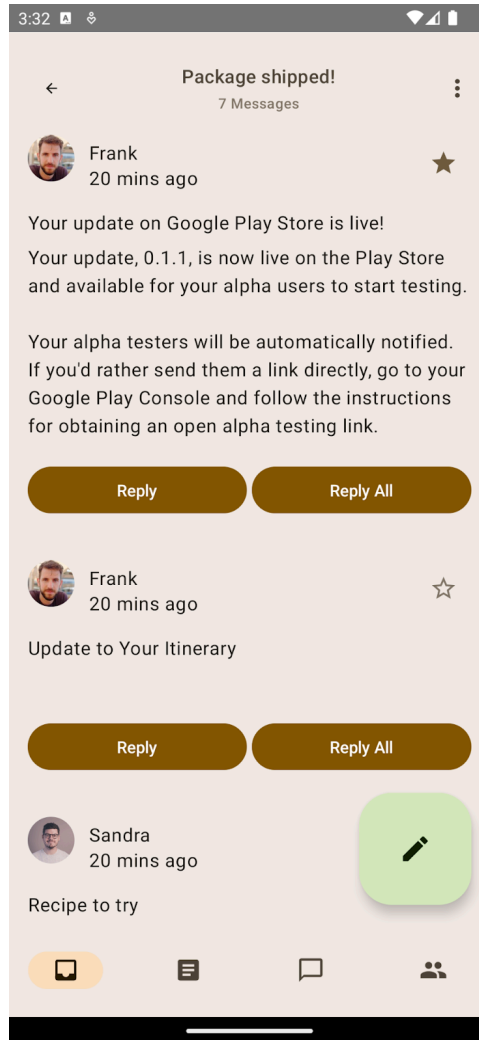


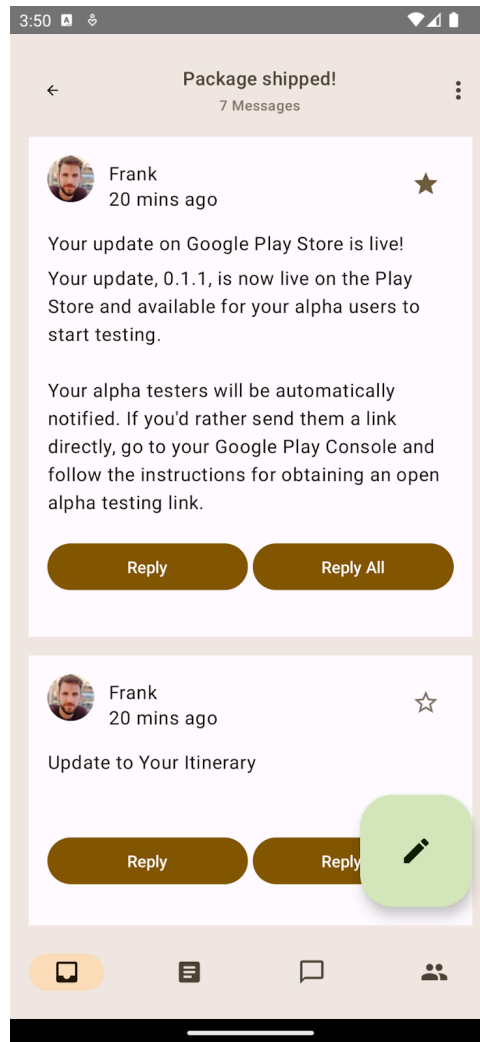


在色調表面使用次要容器顏色醒目顯示清單項目。

詳細資料清單項目的顏色

現在，您已設定主畫面的主題。點選任一電子郵件名單項目，即可查看詳細資料頁面。





清單項目未套用主題的預設詳細資料頁面 (左圖)。

已套用背景主題設定的詳細資料清單項目 (右圖)。

清單項目未套用任何顏色，因此會改回預設的色調表面顏色。您將為清單項目套用背景顏色以做出區別，並加上邊框間距，以便為背景周圍提供間距。

ResponseEmailThreadItem.kt

```
@Composable
fun ReplyEmailThreadItem(
    email: Email,
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier
            .fillMaxWidth()
            .padding(16.dp)
            .background(MaterialTheme.colorScheme.background)
            .padding(20.dp)
    ) {
        // List item content
    }
}
```

您可以看到，只需要提供背景，就能明確區分色調表面和清單項目。

注意：在 Material 3 中，背景顏色在整個主題設定中都維持一致，但表面顏色則不會保持不變。表面顏色會根據表面的高度取得主顏色的色調。

現在您已透過正確的顏色角色及用法，為主畫面和詳細資料頁面設定主題。讓我們來瞭解如何利用動態色彩，讓應用程式提供更符合個人需求且一致的使用體驗。

5. 在應用程式中新增動態色彩

動態色彩是 Material 3 的關鍵部分，其中演算法會從使用者的桌布產生自訂色彩，並套用至其應用程式和系統 UI。

動態主題設定可讓應用程式更貼近個人需求，還能透過系統主題帶給使用者連貫且流暢的體驗。

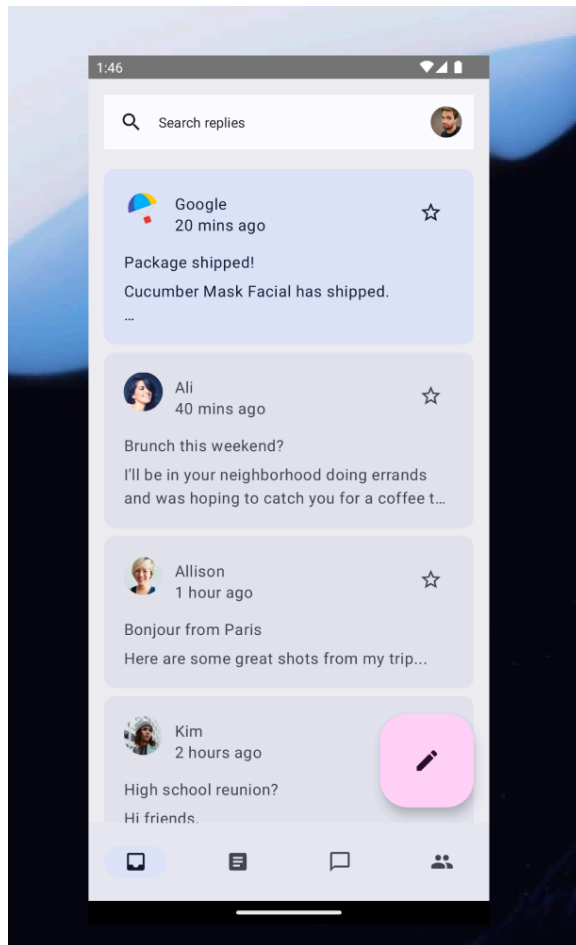
動態色彩適用於 Android 12 以上版本。如有動態色彩可用，您可以使用 `dynamicDarkColorScheme()` 或 `dynamicLightColorScheme()` 設定動態色彩配置。如果沒有，您應改回使用預設的淺色或深色 `ColorScheme`。

將 `Theme.kt` 檔案中 `AppTheme` 函式的程式碼取代為下列程式碼：

Theme.kt

```
@Composable
fun AppTheme(
    useDarkTheme: Boolean = isSystemInDarkTheme(),
    content: @Composable () -> Unit
) {
    val context = LocalContext.current
    val colors = when {
        (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) -> {
            if (useDarkTheme) dynamicDarkColorScheme(context)
            else dynamicLightColorScheme(context)
        }
        useDarkTheme -> DarkColors
        else -> LightColors
    }

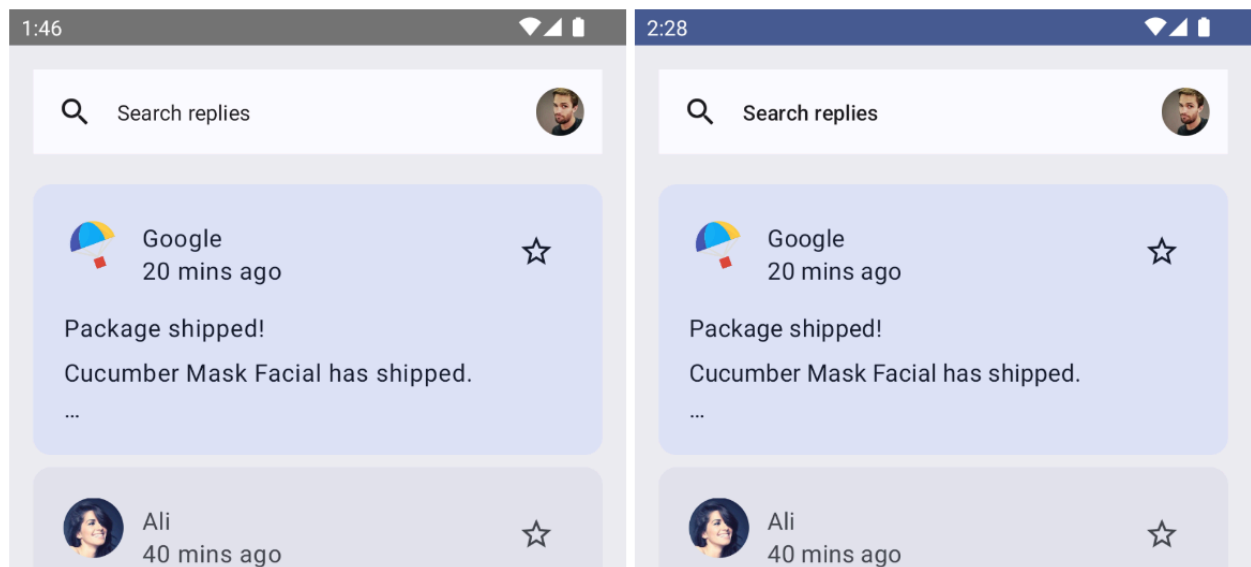
    MaterialTheme(
        colorScheme = colors,
        content = content
    )
}
```



從 Android 13 桌布中擷取的動態主題。

現在執行應用程式，您應該會看到採用的是預設 Android 13 桌布的動態主題設定。

您可能也希望狀態列能根據應用程式主題設定採用的色彩配置，動態調整樣式。



未套用狀態列顏色的應用程式 (左圖)。

已套用狀態列顏色的應用程式 (右圖)。

如要根據主題的主要顏色更新狀態列顏色，請在 `AppTheme` 可組合函式的色彩配置選項後方新增狀態列顏色：

Theme.kt

```

@Composable
fun AppTheme(
    useDarkTheme: Boolean = isSystemInDarkTheme(),
    content: @Composable () -> Unit
) {

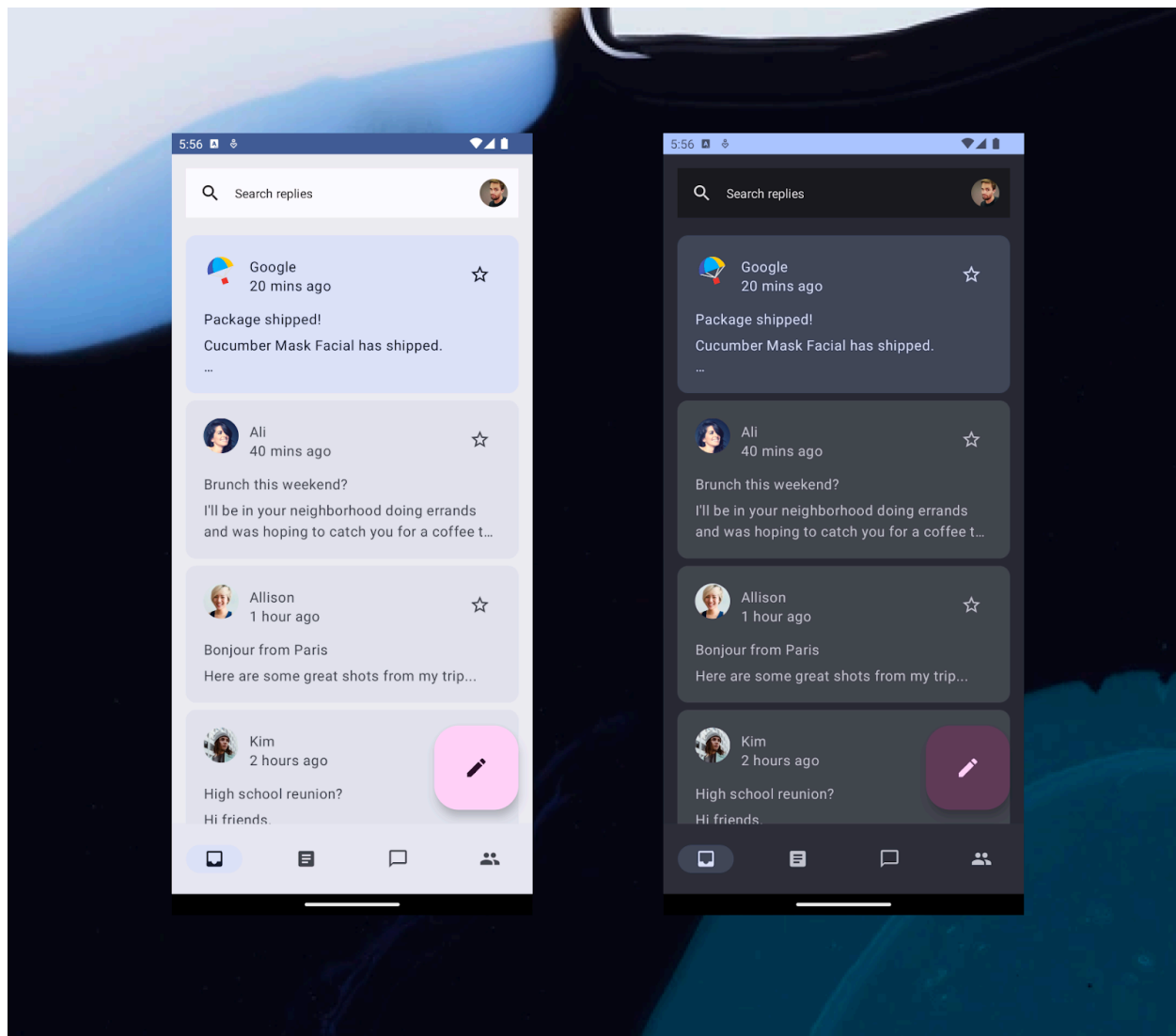
    // color scheme selection code

    // Add primary status bar color from chosen color scheme.
    val view = LocalView.current
    if (!view.isInEditMode) {
        SideEffect {
            val window = (view.context as Activity).window
            window.statusBarColor = colors.primary.toArgb()
            WindowCompat
                .getInsetsController(window, view)
                .isAppearanceLightStatusBars = useDarkTheme
        }
    }

    MaterialTheme(
        colorScheme = colors,
        content = content
    )
}

```

執行應用程式時，您應該會看到採用原色主題設定的狀態列。您也可以變更系統深色主題，藉此嘗試使用淺色和深色動態主題設定。



套用 Android 13 預設桌布的動態淺色 (左側) 和深色 (右側) 主題。

目前為止，您已經為應用程式套用顏色，強化了應用程式的外觀。不過，您可以看到應用程式中所有文字的大小都相同，因此您現在可以在應用程式中新增字體排版。

6. 字體排版

Material Design 3 定義了輸入比例。此外，命名和分組功能也經過簡化，包括顯示、大標題、標題、內文和標籤 (皆有大、中和小三種尺寸)。

Display Large	Roboto 57/64 . 0
Display Medium	Roboto 45/52 . 0
Display Small	Roboto 36/44 . 0
Headline Large	Roboto 32/40 . 0
Headline Medium	Roboto 28/36 . 0
Headline Small	Roboto 24/32 . 0
Title Large	NEW - Roboto Medium 22/28 . 0
Title Medium	Roboto Medium 16/24 . +0.15
Title Small	Roboto Medium 14/20 . +0.1
Body Large	Roboto 16/24 . +0.5
Body Medium	Roboto 14/20 . +0.25
Body Small	Roboto 12/16 . +0.4
Label Large	Roboto Medium 14/20 . +0.1
Label Medium	Roboto Medium 12/16 . +0.5
Label Small	NEW - Roboto Medium 11/16 . +0.5

Material 3 輸入比例。

定義字體排版

Compose 提供 M3 [Typography](#) 類別，配合現有的 [TextStyle](#) 和 [font-related](#) 類別，可建立 Material 3 的輸入比例型式：

字體排版建構函式提供每種樣式的預設值，因此您可以略過任何不想自訂的參數。詳情請參閱字體排版的[樣式和預設值](#)。

您將在應用程式內使用五種字體排版樣式：`headlineSmall`、`titleLarge`、`bodyLarge`、`bodyMedium` 和 `labelMedium`。這些樣式涵蓋了主畫面和詳細資料畫面。



螢幕畫面展示標題、標籤和內文樣式的字體排版用法。

接著，前往 `ui/theme` 套件並開啟 `Type.kt`。新增以下程式碼，以提供您實作的部分文字樣式，而非使用預設值：

Type.kt

```

val typography = Typography(
    headlineSmall = TextStyle(
        fontWeight = FontWeight.SemiBold,
        fontSize = 24.sp,
        lineHeight = 32.sp,
        letterSpacing = 0.sp
    ),
    titleLarge = TextStyle(
        fontWeight = FontWeight.Normal,
        fontSize = 18.sp,
        lineHeight = 28.sp,
        letterSpacing = 0.sp
    ),
    bodyLarge = TextStyle(
        fontWeight = FontWeight.Normal,
        fontSize = 16.sp,
        lineHeight = 24.sp,
        letterSpacing = 0.15.sp
    ),
    bodyMedium = TextStyle(
        fontWeight = FontWeight.Medium,
        fontSize = 14.sp,
        lineHeight = 20.sp,
        letterSpacing = 0.25.sp
    ),
    labelMedium = TextStyle(
        fontWeight = FontWeight.SemiBold,
        fontSize = 12.sp,
        lineHeight = 16.sp,
        letterSpacing = 0.5.sp
    )
)

```

現在您已定義字體排版。若要將其新增至主題，請將其傳遞至 `AppTheme` 中的 `MaterialTheme()` 可組合函式：

Theme.kt

```

@Composable
fun AppTheme(
    useDarkTheme: Boolean = isSystemInDarkTheme(),
    content: @Composable() () -> Unit
) {
    // dynamic theming content

    MaterialTheme(
        colorScheme = colors,
        typography = typography,
        content = content
    )
}

```

使用字體排版

和顏色一樣，您可以使用 `MaterialTheme.typography` 存取目前主題的字體排版樣式。這麼做可為您提供字體排版示例，在 `Type.kt` 中使用所有已定義的字體排版。

```
Text(
    text = "Hello M3 theming",
    style = MaterialTheme.typography.titleLarge
)

Text(
    text = "you are learning typography",
    style = MaterialTheme.typography.bodyMedium
)
```

您的產品不一定要使用 Material Design 輸入比例的 15 個預設樣式。本程式碼研究室中選擇了五種大小，其餘樣式則省略。

您尚未對 `Text()` 可組合函式套用字體排版，因此所有文字會改回預設的 `Typography.bodyLarge`。

主畫面清單的字體排版

接下來，請將字體排版套用至 `ui/components/ReplyEmailListItem.kt` 中的 `ReplyEmailListItem` 函式，以便區分標題和標籤：

ReplyEmailListItem.kt

```
Text(
    text = email.sender.firstName,
    style = MaterialTheme.typography.labelMedium
)

Text(
    text = email.createdAt,
    style = MaterialTheme.typography.labelMedium
)

Text(
    text = email.subject,
    style = MaterialTheme.typography.titleLarge,
    modifier = Modifier.padding(top = 12.dp, bottom = 8.dp),
)

Text(
    text = email.body,
    maxLines = 2,
    style = MaterialTheme.typography.bodyLarge,
    overflow = TextOverflow.Ellipsis
)
```



Search replies



Google
20 mins ago



Package shipped!
Cucumber Mask Facial has shipped.
...



Ali
40 mins ago



Brunch this weekend?
I'll be in your neighborhood doing errands and
was hoping to catch you for a coffee this Sat...



Allison
1 hour ago



Bonjour from Paris
Here are some great shots from my trip...

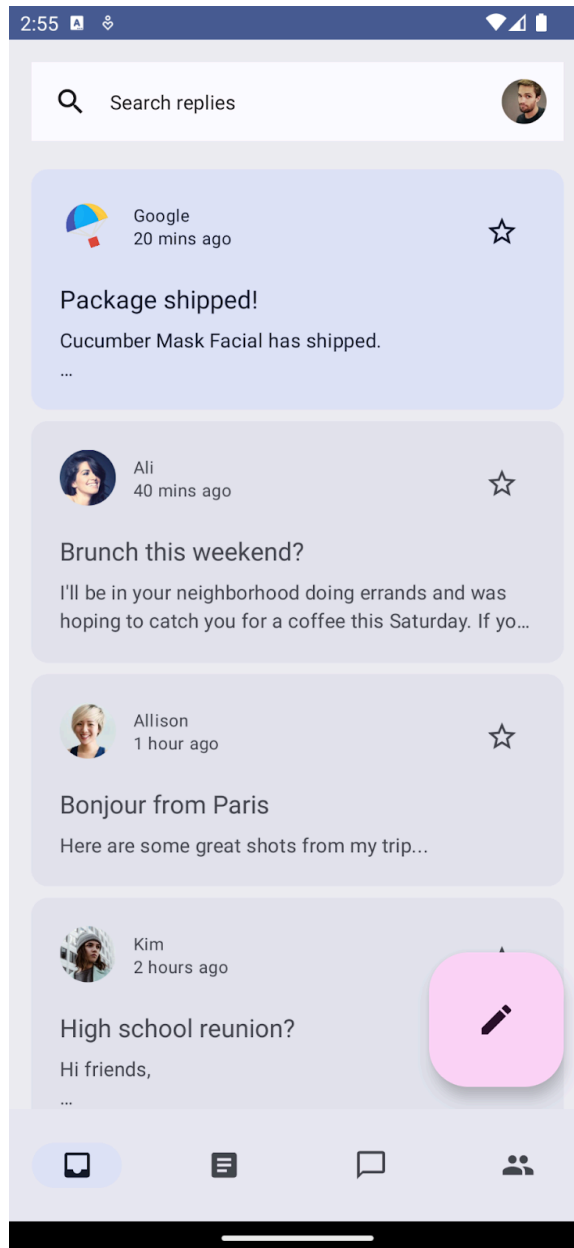


Kim
2 hours ago



High school reunion?
Hi friends.





未套用字體排版的主畫面 (左圖)。

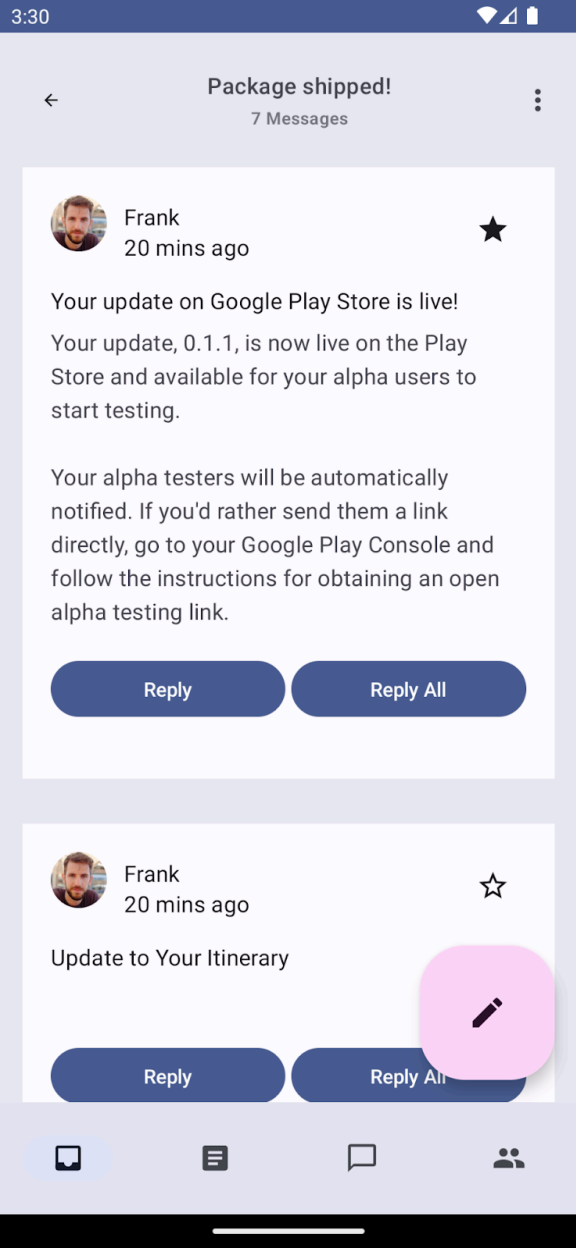
已套用字體排版的主畫面 (右圖)。

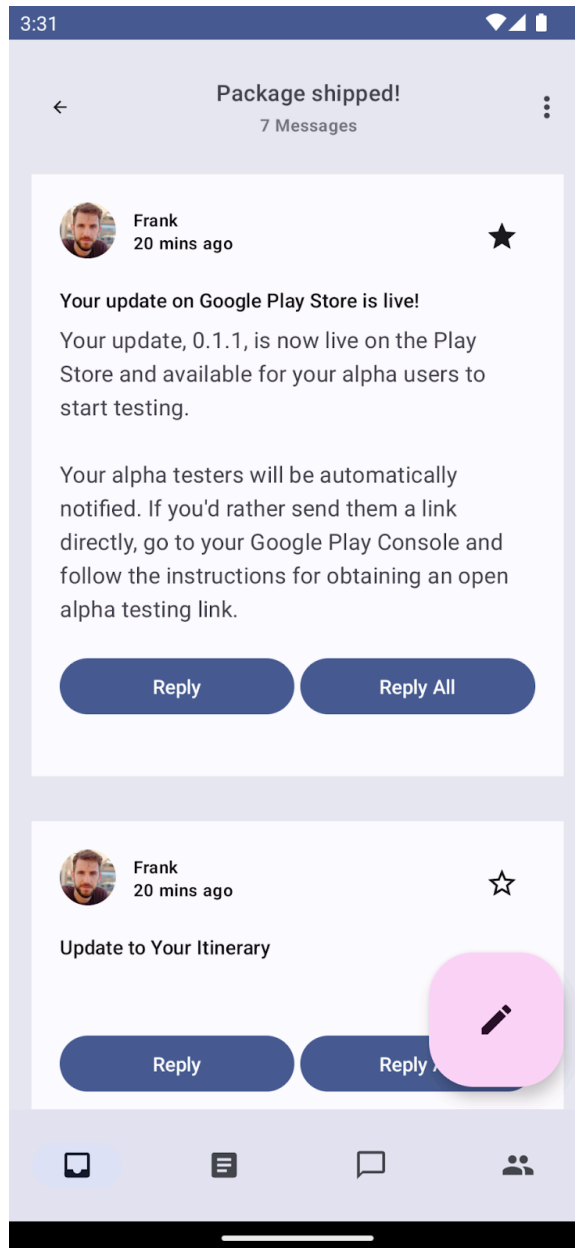
詳細資料清單的字體排版

同樣地，您可以更新 `ui/components/ReplyEmailThreadItem.kt` 中 `ReplyEmailThreadItem` 的所有文字可組合函式，在詳細資料畫面中加入字體排版：

ResponseEmailThreadItem.kt

```
Text(  
    text = email.sender.firstName,  
    style = MaterialTheme.typography.labelMedium  
)  
  
Text(  
    text = stringResource(id = R.string.twenty_mins_ago),  
    style = MaterialTheme.typography.labelMedium  
)  
  
Text(  
    text = email.subject,  
    style = MaterialTheme.typography.bodyMedium,  
    modifier = Modifier.padding(top = 12.dp, bottom = 8.dp),  
)  
  
Text(  
    text = email.body,  
    style = MaterialTheme.typography.bodyLarge,  
    color = MaterialTheme.colorScheme.onSurfaceVariant  
)
```





未套用字體排版的詳細資料畫面 (左圖)。

已套用字體排版的詳細資料畫面 (右圖)。

自訂字體排版

有了 Compose，您就可以輕鬆自訂文字樣式或提供自訂字型。您可以修改 `TextStyle` 來自訂字型類型、字型系列以及字母間距等。

如果變更 `theme/Type.kt` 檔案中的文字樣式，使用該樣式的所有元件也會一併變更。

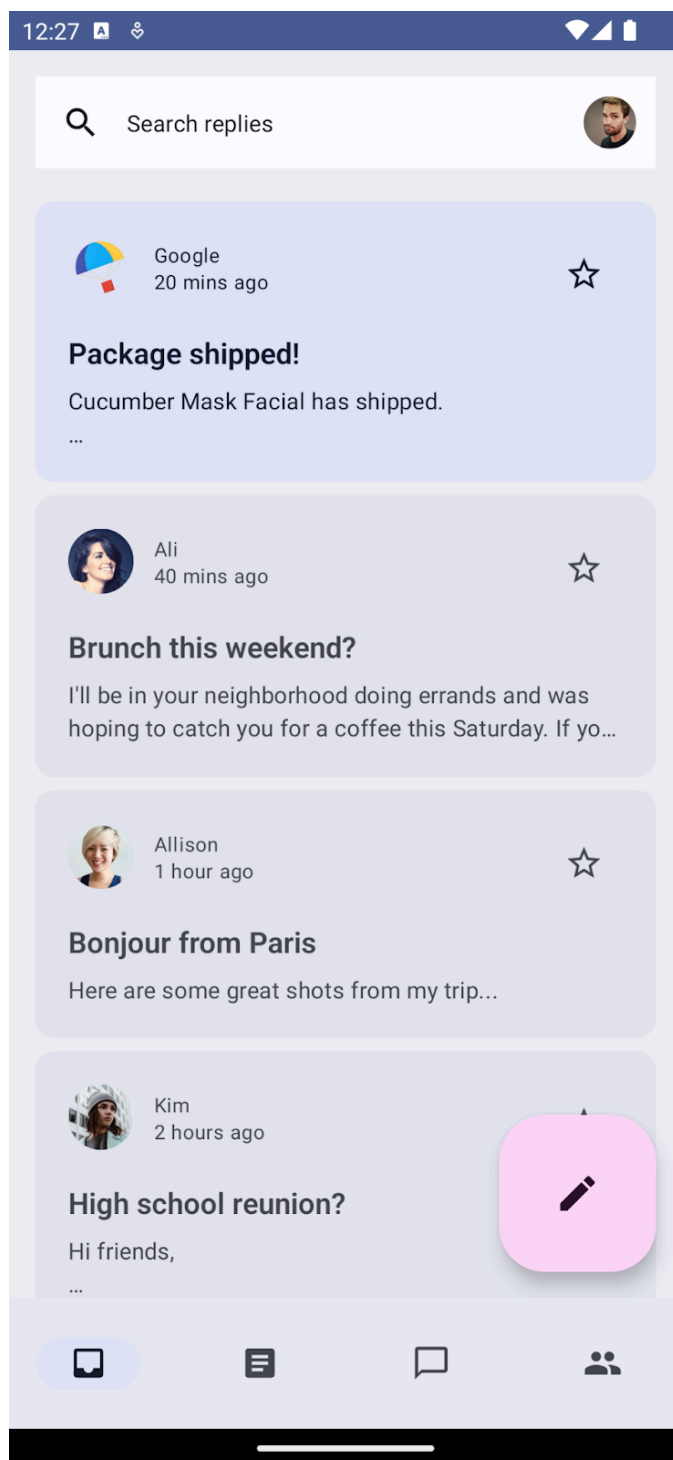
將 `titleLarge` 的 `fontWeight` 更新為 `SemiBold`，並將 `lineHeight` 更新為 `32.sp`，這會用於清單項目中的主旨。這麼做可讓主旨更加顯眼，並與其他字體排版明確區分。

Type.kt

```

...
titleLarge = TextStyle(
  fontWeight = FontWeight.SemiBold,
  fontSize = 18.sp,
  lineHeight = 32.sp,
  letterSpacing = 0.0.sp
),
...

```



對主旨文字套用自訂字體排版。

7. 形狀

Material 表面可用不同形狀顯示。形狀可用於吸引注意、識別元件、傳達狀態，以及呈現品牌風格。

定義形狀

Compose 提供含有擴展參數的 `Shapes` 類別，可實作新的 M3 形狀。M3 形狀比例與[輸入比例](#)類似，可讓您在 UI 中呈現多種形狀。

形狀比例分為不同的形狀大小：

- 特小
- 小
- 中
- 大
- 特大

根據預設，每個形狀都有可覆寫的預設值。您將使用中型形狀修改應用程式的清單項目，但也可以宣告其他形狀。在 `ui/theme` 套件中新建名為 `Shape.kt` 的檔案，然後加入形狀的程式碼：

Shape.kt

```
package com.example.reply.ui.theme

import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.Shapes
import androidx.compose.ui.unit.dp

val shapes = Shapes(
    extraSmall = RoundedCornerShape(4.dp),
    small = RoundedCornerShape(8.dp),
    medium = RoundedCornerShape(16.dp),
    large = RoundedCornerShape(24.dp),
    extraLarge = RoundedCornerShape(32.dp)
)
```

定義 `shapes` 後，請將其傳遞至 M3 `MaterialTheme`，方法與顏色和字體排版相同：

Theme.kt

```
@Composable
fun AppTheme(
    useDarkTheme: Boolean = isSystemInDarkTheme(),
    content: @Composable() () -> Unit
) {
    // dynamic theming content

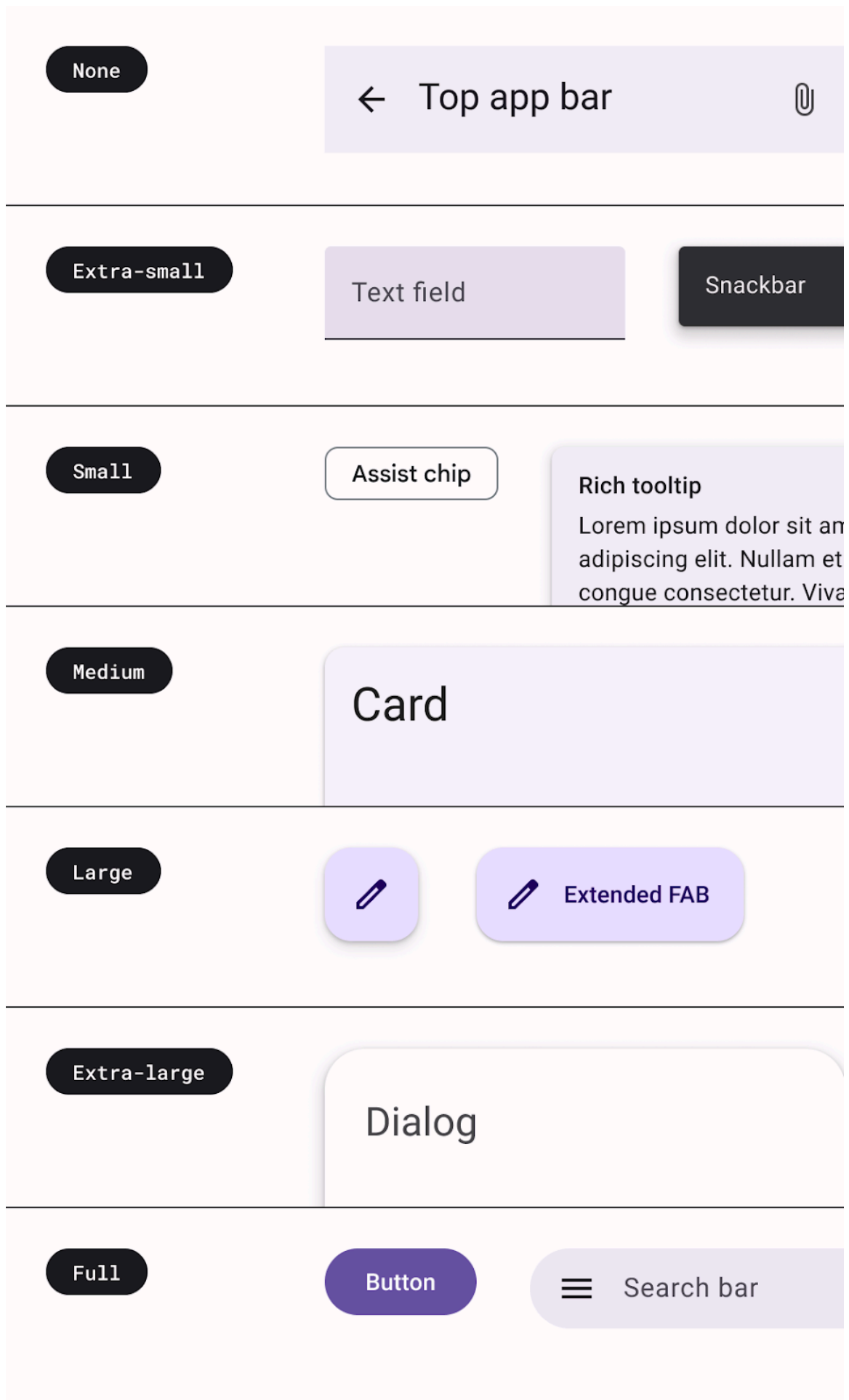
    MaterialTheme(
        colorScheme = colors,
        typography = typography,
        shapes = shapes
        content = content
    )
}
```

使用形狀

就像顏色和字體排版一樣，您可以使用 `MaterialTheme.shape` 將形狀套用至 Material 元件，這樣就能透過 `Shape` 例項存取 Material 形狀。

許多 Material 元件已套用預設形狀，但您可以透過可用版位來提供自己的形狀，並套用至元件。

```
Card(shape = MaterialTheme.shapes.medium) { /* card content */ }
FloatingActionButton(shape = MaterialTheme.shapes.large) { /* fab content */}
```



使用不同形狀的 Material 元件對應。

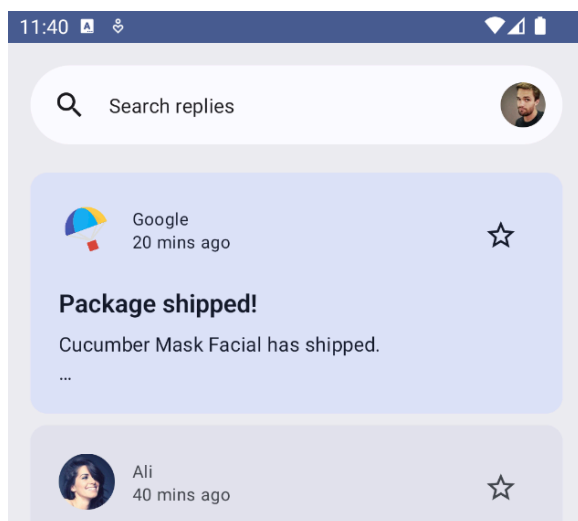
您可以查看形狀相關[文件](#)，瞭解所有元件的形狀對應。

在 Compose 中，您還可以使用另外兩種形狀：`RectangleShape` 和 `CircleShape`。矩形沒有框線半徑，圓形則會顯示完整的圓形邊緣。

您也能使用可接受形狀的 `Modifiers` 將形狀套用至元件，例如 `Modifier.clip`、`Modifier.background` 和 `Modifier.border`。

應用程式列形狀

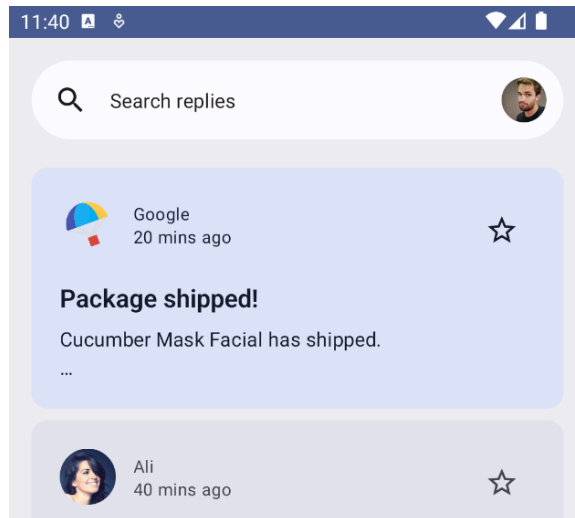
我們希望應用程式列採用圓角背景：



`TopAppBar` 使用的是有背景顏色的 `Row`。如要套用圓角背景，請將 `CircleShape` 傳入背景修飾符，以定義背景形狀：

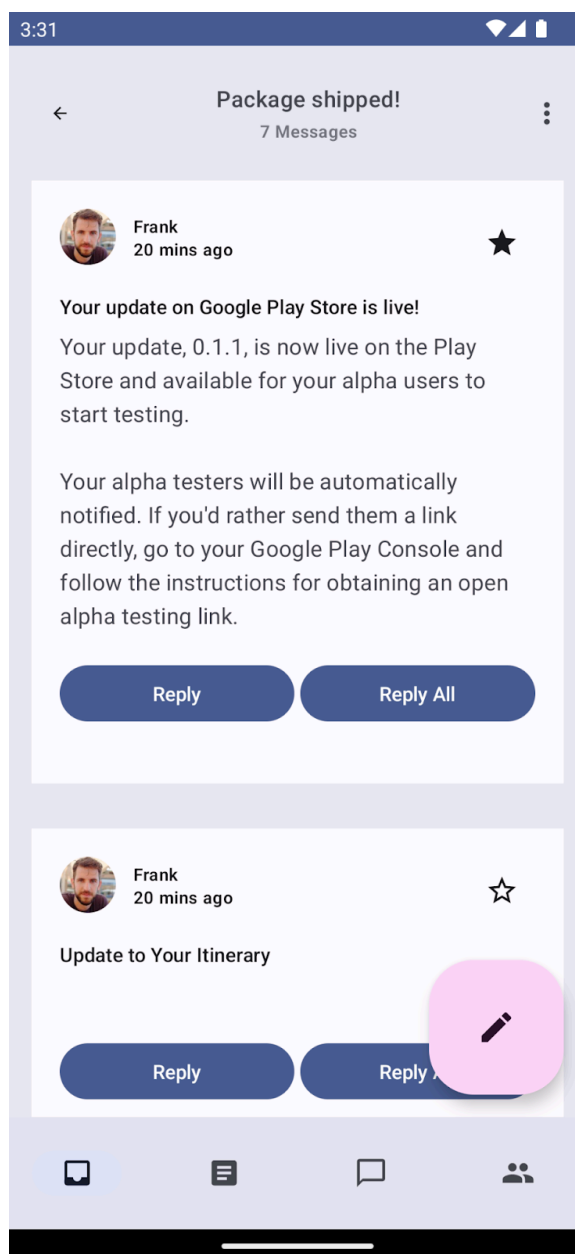
ResponseAppBar.kt

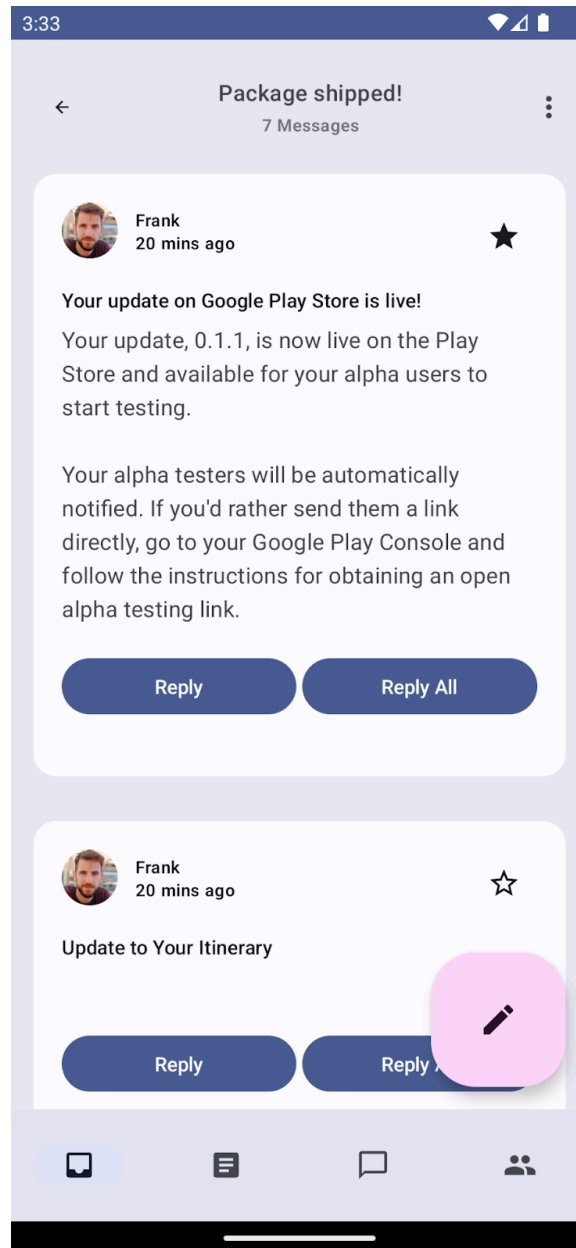
```
@Composable
fun ReplySearchBar(modifier: Modifier = Modifier) {
    Row(
        modifier = modifier
            .fillMaxWidth()
            .padding(16.dp)
            .background(
                MaterialTheme.colorScheme.background,
                CircleShape
            ),
        verticalAlignment = Alignment.CenterVertically
    ) {
        // Search bar content
    }
}
```



詳細資料清單項目的形狀

根據預設，主畫面使用的是採用 `Shape.Medium` 的資訊卡。但在詳細資料頁面中，您改用了含有背景顏色的資料欄。統一清單的外觀時，請為清單套用中等形狀。





詳細資料清單項目的資料欄，左圖的清單項目沒有形狀，右圖的清單上有中等形狀。

ResponseEmailThreadItem.kt


```
@Composable
fun ReplyEmailThreadItem(
    email: Email,
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier
            .fillMaxWidth()
            .padding(8.dp)
            .background(
                MaterialTheme.colorScheme.background,
                MaterialTheme.shapes.medium
            )
            .padding(16.dp)

    ) {
        // List item content
    }
}
```

現在，執行應用程式會看到形狀為 `medium` 的詳細資料畫面清單項目。

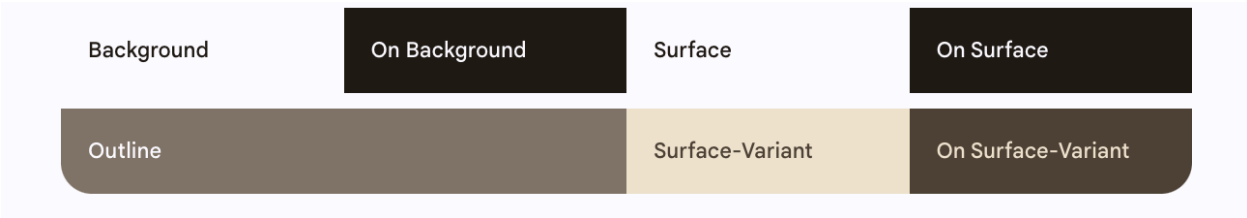
8. 強調效果

UI 中的強調效果可讓某些內容比起其他內容更加顯眼，例如想區分標題和子標題的情況。M3 中的強調效果會使用各種顏色變化及其色彩組合。有兩種方法可新增強調效果：

- 1. 使用表面、surface-variant 和背景，搭配已擴充 M3 色彩系統的 on-surface 和 on-surface-variants 顏色。

舉例來說，表面可與 on-surface-variant 搭配使用，surface-variant 可與 on-surface 搭配使用，藉此提供不同程度的強調效果。

表面變化版本也可以與強調色搭配使用，這麼做的顏色與 on-accent 顏色相比較不顯眼，但依然很容易辨識，也符合對比度。

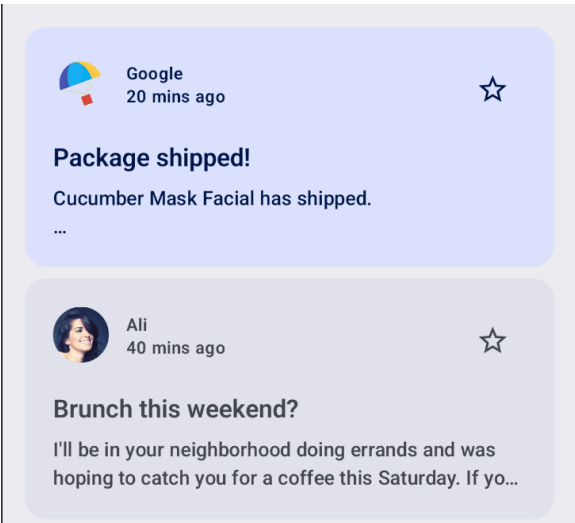


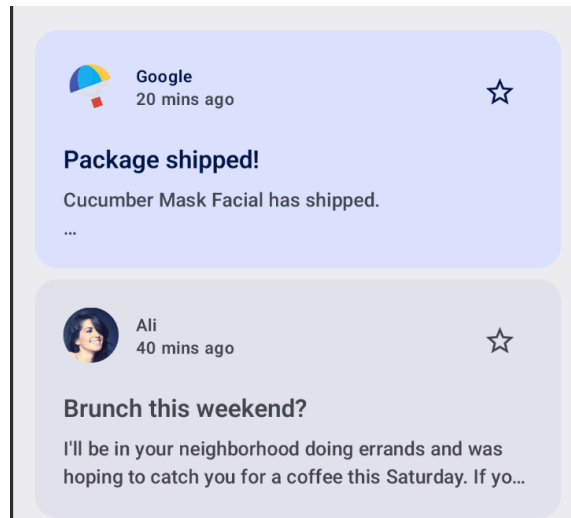
表面、背景和表面變化版本的顏色角色。

- 2. 為文字使用不同的字型粗細。如「字體排版」一節所述，您可以為輸入比例提供自訂粗細，以達到不同的強調效果。

接下來，請使用表面變化版本更新 `ReplyEmailListItem.kt`，藉此呈現強調效果的差異。根據預設，資訊卡內容的預設顏色會採用背景顏色。

您要將時間文字與內文的可組合函式顏色更新為 `onSurfaceVariant`。相較於預設會套用至主旨及標題文字可組合函式的 `onContainerColors`，這種做法會降低強調效果。





時間和內文文字採用與主旨和標題相同的強調效果 (左圖)。

相較於主旨和標題，時間和內文的強調效果較弱 (右圖)。

ReplyEmailListItem.kt

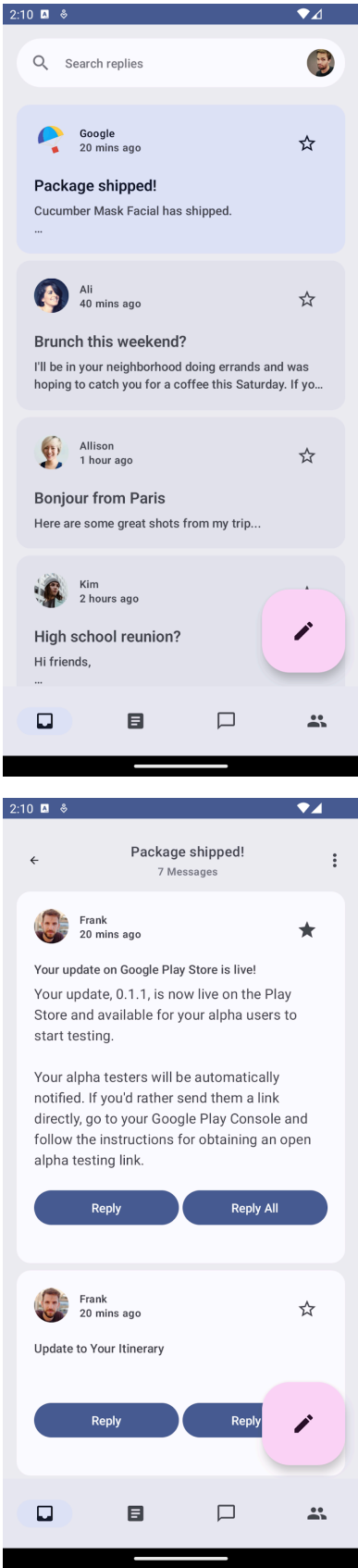
```
Text(  
    text = email.createdAt,  
    style = MaterialTheme.typography.labelMedium,  
    color = MaterialTheme.colorScheme.onSurfaceVariant  
)  
  
Text(  
    text = email.body,  
    maxLines = 2,  
    style = MaterialTheme.typography.bodyLarge,  
    color = MaterialTheme.colorScheme.onSurfaceVariant  
    overflow = TextOverflow.Ellipsis  
)
```

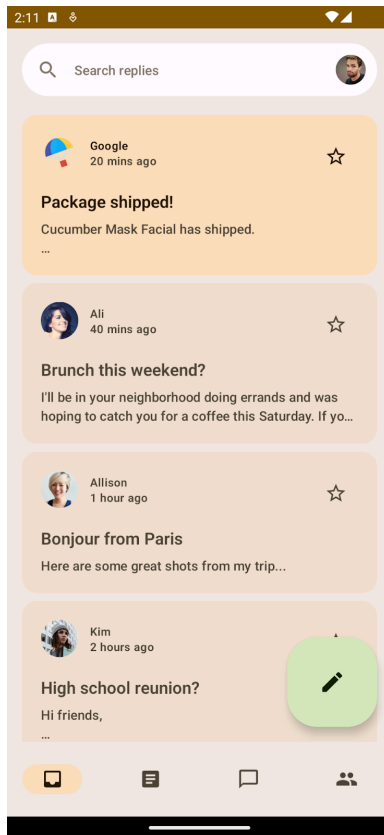
針對背景為 `secondaryContainer` 的重要電子郵件資訊卡，所有文字顏色預設為 `onSecondaryContainer`。其他電子郵件的背景為 `surfaceVariant`，因此所有文字顏色預設為 `onSurfaceVariant`。

步驟 9 · 約 1 分鐘

9. 恭喜

恭喜！您已成功完成本程式碼研究室！您已透過 Compose 實作 Material Design 主題設定，運用顏色、字體排版、形狀，以及動態色彩來為應用程式設定主題，並且提供個人化體驗。





套用動態色彩和色彩主題的主題設定最終結果。

後續步驟

請參閱 [Compose 課程](#) 中的其他程式碼研究室。

- [Compose 基本概念](#)
- [Compose 版面配置](#)
- [Compose 中的狀態](#)
- [現有應用程式的 Compose](#)

其他資訊

- [Compose 主題設定指南](#)
- Compose 的 [Material Design 主題設定](#)

範例應用程式

- 套用完整 Material 3 主題設定的 [Reply 範例](#) 應用程式
- 示範動態主題設定的 [JetChat](#)

參考文件

- [Material Design 元件](#)
 - [Material Design 主題](#)
 - [色彩配置](#)、[形狀](#)和[字體排版](#)
-